Исследовательская работа

на тему «Применение методов статистического анализа данных с использованием языка программирования на примере «Линейной регрессии, сравнения МНК и метода градиентного спуска»

Выполнила:

Федорина Дарья

СОДЕРЖАНИЕ

Задание на курсовую работу.....3
1 Формулирование требований .....**Ошибка! Закладка не определена.**
1.1 Введение.....**Ошибка! Закладка не определена.**
1.2 Цель.....**Ошибка! Закладка не определена.**
1.3 Задачи .....**Ошибка! Закладка не определена.**
2. Описание и подготовка исходных данных .....**Ошибка! Закладка не определена.**
2.1 Описание данных .....**Ошибка! Закладка не определена.**
2.2 Характеристики выборки.....**Ошибка! Закладка не определена.**
2.3 Предварительная обработка данных .....**Ошибка! Закладка не определена.**
3. Основной анализ данных .....**Ошибка! Закладка не определена.**
3.1 Предварительная обработка данных .....**Ошибка! Закладка не определена.**
3.2 Статистические метрики.....**Ошибка! Закладка не определена.**
3.3 Распределения данных.....**Ошибка! Закладка не определена.**
4. Моделирование и прогнозирование.....31
4.1 Прогнозирование временных рядов с помощью ARIMA-моделей.....31
5. Аналитические выводы 35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 39

Задание на курсовую работу

по дисциплине

Статистические методы анализа данных в принятии решений

Студентке группы ББИ-23-6 Федориной Д.Д

(номер группы) (фамилия, имя, отчество)

Тема курсовой работы: «Применение методов статистического анализа данных с использованием языка программирования Python / MS Excel на примере «…»

Методы исследования: анализ выбросов, нормализация данных, анализ статистических показателей, графический анализ, корреляционный и регрессионный анализ, проверка гипотез, метод градиентного спуска, анализ временных рядов

Инструментальные средства моделирования: Python / Excel

№	Этапы выполнения курсовой работы	Срок выполнения
1	Получение задания на курсовую работу. Подготовка введения. Описание целей, задач, выбранного инструмента	10.02–23.02.2025
2	Подготовка раздела «Описание и подготовка исходных данных»	24.02–09.03.2025
3	Подготовка раздела «Статистический анализ данных»	10.03–30.03.2025
4	Подготовка раздела «Моделирование и прогнозирование» (2 варианта)	31.03–20.04.2025
5	Формулирование выводов	21.04–27.04.2025
6	Подготовка презентации, защита	28.04–25.05.2025

Дата выдачи задания 10.02.25 – 23.02.25

Дата сдачи курсовой работы 28.04.25 – 05.05.25

Дата защиты 12.05.24 – 31.05.24

Руководитель курсовой работы Исаева Н. А. / Котеленко С.А. / Литвяк В.С.

Задание приняла студентка
группы ББИ- 23-6
Федорина Дарья Денисовна

1 Формирование требований

1.1 Введение

В условиях активного развития цифровых технологий и роста объемов информации особенно важной становится задача анализа данных, способствующего принятию обоснованных решений. Музыкальные стриминговые сервисы, социальные платформы и рекомендательные системы ежедневно формируют и обрабатывают огромные массивы пользовательских данных. В этой связи востребованы методы, позволяющие выявлять скрытые закономерности в поведении пользователей и характеристиках контента. Анализ музыкальных треков с использованием статистических методов, таких как корреляционный и регрессионный анализ, позволяет не только понять, какие параметры влияют на популярность треков, но и использовать полученные модели для прогноза и рекомендации. Работа опирается на реальные данные и сочетает классические подходы (метод наименьших квадратов) с современными алгоритмами, такими как градиентный спуск. Практическая значимость исследования заключается в возможности применения полученных результатов для повышения эффективности цифровых сервисов, в том числе рекомендательных систем.

1.2 Цель

Изучение и применение методов статистического анализа для выявления факторов, влияющих на популярность музыкальных треков, с последующим построением и сравнением моделей линейной регрессии на языке Python.

1.3 Задачи

— Провести предварительный анализ данных: изучение структуры, очистка от пропусков и выбросов.

— Выполнить описательную статистику и визуализацию распределений.

— Исследовать корреляции между музыкальными характеристиками и популярностью треков.

— Построить линейную регрессионную модель с использованием метода наименьших квадратов.

— Реализовать градиентный спуск для обучения линейной модели.

— Сравнить результаты моделей по коэффициентам, метрике RMSE и графикам остатков.

— Сформулировать аналитические выводы и рекомендации по улучшению качества моделей.

— Обосновать практическое применение полученных навыков в реальных аналитических задачах.

1.4. Описание используемых источников

Для реализации методов линейной регрессии и сравнительного анализа МНК и градиентного спуска в работе используется язык программирования Python. Выбор обусловлен следующими преимуществами:

- Широкие возможности для статистического анализа и машинного обучения
- Наличие специализированных библиотек для научных вычислений
- Открытость и кроссплатформенность

Основные используемые библиотеки:

- NumPy: фундаментальная библиотека для научных вычислений, предоставляющая поддержку многомерных массивов и матричных операций. Содержит математические функции для операций линейной алгебры, что критически важно для реализации метода наименьших квадратов. Позволяет эффективно выполнять векторные операции и матричные преобразования.
- Pandas: основная библиотека для работы со структурированными данными. Основные структуры данных - DataFrame и Series - обеспечивают удобный интерфейс для загрузки, предварительной обработки и анализа данных. Полезна при работе с пропущенными значениями и категориальными переменными.
- Matplotlib/Seaborn: библиотеки для визуализации данных. Matplotlib предоставляет низкоуровневые функции для построения различных типов графиков, в то время как Seaborn, построенный на его основе, предлагает более высокоуровневый интерфейс для статистической визуализации. Используются для отображения регрессионных зависимостей и процессов оптимизации.
- Scikit-learn: одна из основных библиотек машинного обучения. Содержит готовые реализации алгоритмов регрессии, включая линейную регрессию, что позволяет проверять корректность собственных реализаций. Также предоставляет набор метрик для оценки качества моделей.
- Statsmodels: библиотека для статистического анализа данных. Позволяет получать детальную статистическую информацию о построенных моделях, включая значимость коэффициентов и различные диагностические показатели. Особенно полезна для проверки статистических гипотез.

2. Предварительный анализ исходных данных

Для выполнения курсовой работы был использован набор данных под названием "SpotifyFeatures.csv". Этот файл содержит информацию о музыкальных треках, доступных на платформе Spotify. Данные включают различные характеристики треков, такие как длительность, жанр, популярность, акустические параметры и другие метрики, которые могут быть полезны для анализа.

2.1 Описание источника данных

Формат данных: CSV (Comma-Separated Values)

Количество строк и столбцов: Данные содержат множество строк (предположительно, несколько тысяч) и несколько десятков столбцов, каждый из которых представляет определенную характеристику трека.

Описание каждого столбца из датасета:

— Genre: Жанр трека

— Artict_name: Имя исполнителя

— Track_name: Название трека

— Track_id: Уникальный идентификатор трека.

— Popularity: Показатель популярности трека (от 0 до 100)

— Acousticness : Мера акустичности трека (от 0.0 до 1.0).

— Danceability : Показатель танцевальности трека (от 0.0 до 1.0). Пример: 0.313.

— Duration_ms: Длительность трека в миллисекундах Energy: Показатель энергии трека, отражающий его интенсивность и динамику.

— Instrumentalness :Мера инструментальности трека (от 0.0 до 1.0).

— Key: Музыкальный ключ трека.

— Liveness: Характеристика, которая отражает наличие «Эффекта живого выступления»

— Loudness: Громкость трека в децибелах (dB).

— Key: Музыкальный ключ трека.

— Mode: Лад трека(Major или Minor)

— Speechiness :Наличие речевых элементов (от 0.0 до 1.0).

— Tempo :Темп трека в ударах в минуту (BPM).

— Time_Signature: Размер такта

— Valence: Позитивность звучания трека

2.2 Характеристики выборки

Для проведения статистического анализа используется выборка музыкальных композиций в жанре "World", содержащая 100 наблюдений.

В выборке присутствуют как количественные переменные, так и качественные

	genre	artist_name	track_name	track_id	popularity	acousticness	danceability	duration_ms	energy	instrumentalness	key	liveness	loudness	mode	speechiness	tempo	time_signature	valence
count	100	100	100	100	100.00000	100.000000	100.000000	1.000000e+02	100.000000	100.000000	100	100.000000	100.000000	100	100.000000	100.000000	100	100.000000
unique	1	78	100	100	NaN	NaN	NaN	NaN	NaN	NaN	12	NaN	NaN	2	NaN	NaN	3	NaN
top	World	Don Moen	Matchless Love	1fTnfXrSznVsoYw5HwaaOy	NaN	NaN	NaN	NaN	NaN	NaN	C	NaN	NaN	Major	NaN	NaN	4/4	NaN
freq	100	4	1	1	NaN	NaN	NaN	NaN	NaN	NaN	19	NaN	NaN	66	NaN	NaN	79	NaN
mean	NaN	NaN	NaN	NaN	31.30000	0.443807	0.449351	3.181994e+05	0.492804	0.284325	NaN	0.220100	-11.971340	NaN	0.039954	117.572270	NaN	0.337618
std	NaN	NaN	NaN	NaN	4.90001	0.372117	0.163504	1.564856e+05	0.271803	0.393272	NaN	0.221707	7.574503	NaN	0.015142	27.450321	NaN	0.256027
min	NaN	NaN	NaN	NaN	22.00000	0.000143	0.063800	1.088400e+05	0.006350	0.000000	NaN	0.055200	-39.915000	NaN	0.025400	57.395000	NaN	0.033300
25%	NaN	NaN	NaN	NaN	28.00000	0.058025	0.338500	2.342570e+05	0.290750	0.000002	NaN	0.097675	-14.161750	NaN	0.030500	96.058000	NaN	0.127000
50%	NaN	NaN	NaN	NaN	30.00000	0.384500	0.481500	2.814265e+05	0.514000	0.000361	NaN	0.116500	-9.461500	NaN	0.036750	116.908000	NaN	0.267500
75%	NaN	NaN	NaN	NaN	35.00000	0.766500	0.560750	3.786232e+05	0.713750	0.767000	NaN	0.218250	-6.909500	NaN	0.043600	137.810250	NaN	0.492000
max	NaN	NaN	NaN	NaN	45.00000	0.995000	0.782000	1.362467e+06	0.962000	0.974000	NaN	0.972000	-2.184000	NaN	0.130000	183.941000	NaN	0.954000

Рисунок 1. Визуализация в табличном виде количественных и качественных переменных

Количественные переменные представлены метриками музыкальных характеристик со следующими статистическими показателями:

— Популярность (popularity): значения варьируются от 22 до 45 баллов (среднее 31.3 и медиана 30)

— Акустичность (acousticness): диапазон от 0.0001 до 0.9950 (среднее 0.4438, медиана 0.3845)

— Танцевальность (danceability): от 0.0638 до 0.7820 (среднее 0.4494, медиана: 0.4815)

— Продолжительность (duration_ms): от 108 840 мс (1:48) до 1 362 467 мс (22:42) (среднее 318 199.37 мс (5.30 минут), медиана 281426.50**)**

— Энергичность (energy): от 0.0063 до 0.9620 (среднее 0.4928, медиана 0.5140)

— Инструментальность (instrumentalness): от 0 до 0.9740 (среднее 0.2843, медиана 0.0004)

— Громкость (loudness): от -39.9150 dB до -2.1840 dB (среднее -11.9713 dB, медиана -9.4615)

— Темп (tempo): от 57.3950 до 183.9410 BPM (среднее 117.5723 BPM, медиана 116.9080)

— Живость (liveness): от 0.0552 до 0.9720 (среднее 0.2201)

— Наличие речи в треке (speechiness): от 0.0254 до 0.1300 (среднее 0.0399)

— Позитивность звучания (valence): от 0.0333 до 0.9540 (среднее 0.3376)

Качественные переменные включают:

— Жанр (genre): представлен один жанр Word

— Исполнитель(artict_name): 78 уникальных исполнителей, наиболее часто встречающийся – Don Moen в 4 треках

— Название трека (track_name): все значения уникальны- 100 уникальных значений

— ID трека (track_id): уникален для каждого трека

— Тональность (key): 12 категорий (A, A#, B, C, C#, D, D#, E, F, F#, G, G#). Наиболее часто встречающийся – C

— Лад (mode): два значения Major, Minor. Преобладает Major

— Музыкальный размер (time_signature): представлено 3 уникальных размера 3/4, 4/4, 5/4. Чаще всего встречается 4/4

2.3 Предварительная обработка данных

```python
try:
    df = pd.read_excel('data.xlsx')
    print("Данные успешно загружены")
    print(f"Размерность данных: {df.shape[0]} строк, {df.shape[1]} столбцов")
except Exception as e:
    print(f"Ошибка при загрузке файла: {e}")
    exit()
print("\nПервые 5 строк данных:")
print(df.head().to_string())
print("\nИнформация о данных:")
print(df.info())
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
print("\nОсновные статистики для числовых переменных:")
print(df[numeric_cols].describe().round(2).to_string())
print("\nАнализ пропущенных значений:")
missing_values = df.isnull().sum()
missing_percent = (df.isnull().mean() * 100).round(2)
missing_report = pd.DataFrame({
    'Количество пропусков': missing_values,
    'Процент пропусков (%)': missing_percent
})
print(missing_report[missing_report['Количество пропусков'] > 0].to_string())
categorical_cols = df.select_dtypes(include=['object']).columns
print("\nАнализ категориальных переменных:")
for col in categorical_cols:
    print(f"\nСтолбец: {col}")
    print(f"Количество уникальных значений: {df[col].nunique()}")
    print("Топ-5 частых значений:")
    print(df[col].value_counts().head().to_string())
with open('data_analysis_report.txt', 'w', encoding='utf-8') as f:
    f.write("ОТЧЕТ О ПЕРВИЧНОМ АНАЛИЗЕ ДАННЫХ\n")
    f.write(f"Общее количество наблюдений: {len(df)}\n")
    f.write(f"Количество признаков: {len(df.columns)}\n")
    f.write("\nСтатистики числовых переменных:\n")
    f.write(df[numeric_cols].describe().round(2).to_string())
    f.write("\n\nАнализ пропущенных значений:\n")
    f.write(missing_report.to_string())

print("\nОтчет сохранен в файл 'data_analysis_report.txt'")
```

Рисунок 2.Загрузка и первичный осмотр данных

Файл "SpotifyFeatures.csv" был загружен в среду анализа данных Google Colaboratory с помощью специализированной библиотеки pandas. Был проведен первичный осмотр данных для понимания их структуры и выявления возможных проблем, таких как пропущенные значения или некорректные данные. Анализ загруженных данных показал, что выборка SpotifyFeatures содержит 100 музыкальных треков с 18 характеристиками, включая как числовые (popularity, acousticness, danceability и др.), так и категориальные (artist_name, track_name, key и др.) переменные. Проверка на пропущенные значения выявила их полное отсутствие во всех столбцах, что свидетельствует о хорошем качестве исходных данных. Основные статистики числовых переменных демонстрируют значительный разброс значений: параметр popularity варьируется от 22 до 45 баллов (среднее 31.3), продолжительность треков (duration_ms) - от 108840 до 1362467 мс, а показатель loudness имеет диапазон от -39.92 до -2.18 dB. Категориальные переменные обладают ожидаемыми характеристиками: все треки принадлежат жанру "World", столбец artist_name содержит 78 уникальных исполнителей, а track_name и track_id полностью

уникальны для каждого трека. Анализ музыкальных параметров выявил, что наиболее распространенной тональностью является C (19 треков), а размер 4/4 доминирует в выборке (83% случаев). Полученные результаты подтверждают репрезентативность выборки и ее пригодность для дальнейшего статистического анализа и построения прогнозных моделей.

```python
print("Анализ пропущенных значений:")
print("=" * 50)
missing_values = df.isnull().sum()
missing_percent = (df.isnull().mean() * 100).round(2)
missing_report = pd.DataFrame({
    'Количество пропусков': missing_values,
    'Процент пропусков (%)': missing_percent
})
print(missing_report[missing_report['Количество пропусков'] > 0])
if missing_values.sum() > 0:
    df_cleaned = df.dropna()
    print(f"\nУдалено строк с пропусками: {len(df) - len(df_cleaned)}")
    print(f"Новый размер данных: {df_cleaned.shape}")
    df = df_cleaned
else:
    print("\nПропущенных значений не обнаружено, удаление не требуется")
print("\nПроверка после обработки:")
print(f"Осталось пропусков: {df.isnull().sum().sum()}")
```

```
Анализ пропущенных значений:
==================================================
Empty DataFrame
Columns: [Количество пропусков, Процент пропусков (%)]
Index: []

Пропущенных значений не обнаружено, удаление не требуется
```

Рисунок 3. Проверка на пропущенные значения

На рисунке 3 представлен результат выполнения Python-кода для анализа пропущенных значений в датафрейме. Код подсчитывает количество и процент пропусков для каждой переменной. По результату видно, что в данных отсутствуют пропущенные значения, поэтому очистка не требуется

```python
import numpy as np
import matplotlib.pyplot as plt
def find_outliers(df):
    outliers_dict = {}
    for column in df.columns:
        if pd.api.types.is_numeric_dtype(df[column]):
            Q1 = df[column].quantile(0.25)
            Q3 = df[column].quantile(0.75)
            IQR = Q3 - Q1
            lower_bound = Q1 - 1.5 * IQR
            upper_bound = Q3 + 1.5 * IQR
            outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
            outliers_dict[column] = {
                'outliers_count': len(outliers),
                'outliers_percent': f"{(len(outliers)/len(df))*100:.2f}%",
                'min_value': df[column].min(),
                'max_value': df[column].max(),
                'lower_bound': lower_bound,
                'upper_bound': upper_bound,
                'outliers_indices': outliers.index.tolist()
            }
        else:
            value_counts = df[column].value_counts(normalize=True)
            rare_categories = value_counts[value_counts < 0.05].index.tolist()

            if rare_categories:
                outliers = df[df[column].isin(rare_categories)]
                outliers_dict[column] = {
                    'rare_categories_count': len(rare_categories),
                    'outliers_count': len(outliers),
                    'outliers_percent': f"{(len(outliers)/len(df))*100:.2f}%",
                    'rare_categories': rare_categories,
                    'outliers_indices': outliers.index.tolist()
                }
    return outliers_dict
outliers_report = find_outliers(df)
print("Полный отчет по выбросам:")
for col, data in outliers_report.items():
    print(f"\nКолонка: {col}")
    if 'min_value' in data:
        print(f"  Количество выбросов: {data['outliers_count']} ({data['outliers_percent']})")
        print(f"  Границы выбросов: [{data['lower_bound']:.2f}, {data['upper_bound']:.2f}]")
        print(f"  Диапазон значений: [{data['min_value']:.2f}, {data['max_value']:.2f}]")
```

Рисунок 4. Проверка на выбросы-1 часть кода

```python
        print(f"  Диапазон значений: [{data['min_value']:.2f}, {data['max_value']:.2f}]")
    else:
        if 'rare_categories' in data:
            print(f"  Редкие категории: {len(data['rare_categories'])} ({', '.join(map(str, data['rare_categories']))})")
            print(f"  Количество выбросов: {data['outliers_count']} ({data['outliers_percent']})")
numeric_cols = df.select_dtypes(include=['number']).columns
if not numeric_cols.empty:
    plt.figure(figsize=(15, 6))
    df[numeric_cols].boxplot()
    plt.title('Распределение числовых переменных с выбросами')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

Рисунок 5. Проверка на выбросы-2 часть кода

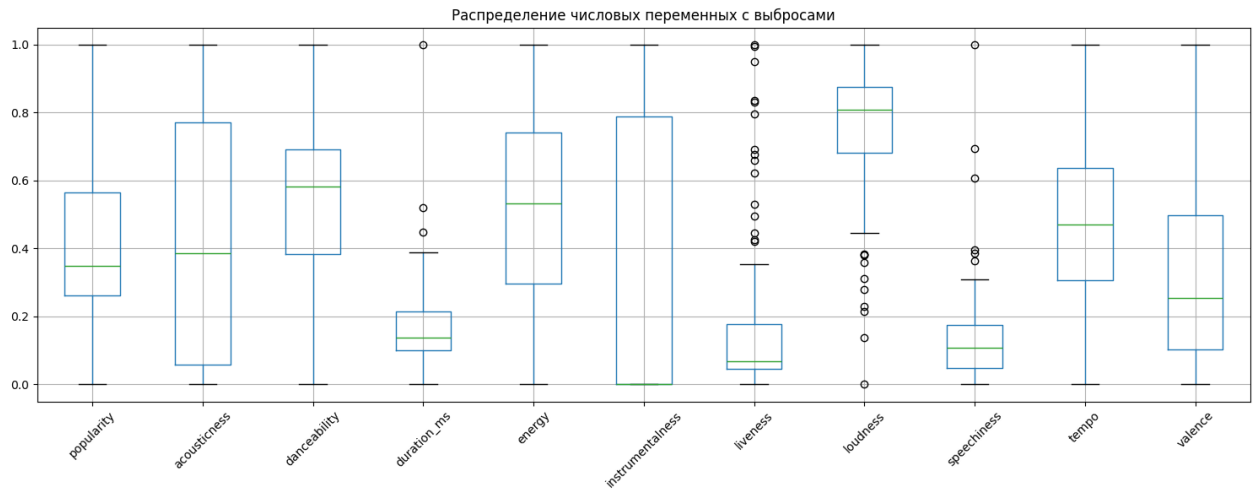

Рисунок 6. Визуализация числовых колонок- Boxplot для всех переменных

На основе проведённого анализа были выявлены выбросы в числовых показателях.

Среди числовых признаков наибольшее количество выбросов наблюдается в переменной liveness — 15%, где значения превышают верхнюю границу в 0.38 и достигают 0.975. В переменной loudness зафиксировано 10% выбросов: значения громкости ниже нижней границы 0.39, что может указывать на технические особенности записи или треки с минимальным уровнем громкости. Также выбросы выявлены в переменной speechiness, где 6% значений превышают верхнюю границу 0.36, что характерно для треков с преобладанием речевых элементов. В переменной duration_ms зафиксировано 3 выброса-3% наблюдений, выходящих за верхнюю границу 0.39, что указывает на длительные композиции.

Остальные числовые переменные, включая popularity, acousticness, danceability, energy, instrumentalness, tempo и valence, не содержат выбросов — все значения находятся в пределах допустимых границ, рассчитанных по методу межквартильного размаха (IQR).

Для идентификации выбросов использовался метод IQR, где значения, выходящие за границы интервала [Q1 - 1.5 * IQR; Q3 + 1.5 * IQR], признаются аномальными. Данный подход является устойчивым к распределениям, не подчиняющимся нормальному закону, и позволяет корректно выявить значимые отклонения. Удаление выбросов особенно актуально для переменных с высокой долей аномалий, таких как liveness и loudness, поскольку они могут искажать результаты последующего анализа и влиять на устойчивость моделей. В то же время выбросы в уникальных категориальных переменных, таких как track_id, не подлежат удалению, так как являются частью уникальной структуры музыкальных данных.

```python
from sklearn.preprocessing  import MinMaxScaler

print("\nНормализация данных:")
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
print("\nДанные после нормализации:")
print(df[numeric_cols].describe().loc[["min", "max"]].round(2))
df.to_excel('processed_data.xlsx', index=False)
print("\nОбработанные данные сохранены в файл 'processed_data.xlsx'")
```

```
Нормализация данных:

Данные после нормализации:
     popularity  acousticness  danceability  duration_ms  energy  \
min         0.0           0.0           0.0          0.0     0.0
max         1.0           1.0           1.0          1.0     1.0

     instrumentalness  liveness  loudness  speechiness  tempo  valence
min               0.0       0.0       0.0          0.0    0.0      0.0
max               1.0       1.0       1.0          1.0    1.0      1.0
```

Рисунок 6. Нормализация данных- код и результат

На рисунке 6 представлен код, который выполняет нормализацию числовых данных методом MinMaxScaler, преобразующим значения всех числовых столбцов в диапазон от 0 до 1. На первом этапе создается объект scaler класса MinMaxScaler, после чего метод fit_transform применяется к выбранным числовым признакам (numeric_cols). После нормализации осуществляется проверка результатов с помощью метода describe(), который позволяет убедиться в том, что минимальные значения столбцов стали равны 0.0, а максимальные — 1.0. Заключительным этапом является сохранение обработанных данных в Excel-файл под названием 'processed_data.xlsx' с отключённым индексом (index=False).

Выбор метода MinMaxScaler обоснован необходимостью привести переменные к единому масштабу, учитывая то, что признаки имеют разные диапазоны и единицы измерения. В отличие от других методов масштабирования, MinMaxScaler сохраняет исходную форму распределения данных, не искажая относительные различия между значениями.

```python
def preprocess_data(dataframe, exclude_cols=None):
    if exclude_cols is None:
        exclude_cols = ['track_id', 'track_name', 'artist_name', 'genre']
    stats = {
        'rows_start': len(dataframe),
        'duplicates_dropped': 0,
        'null_handling': {},
        'outliers_dropped': 0,
        'invalid_replaced': 0,
        'rows_with_na_dropped': 0,
        'rows_final': len(dataframe)
    }
    df_proc = dataframe.copy()
    pre_drop_len = len(df_proc)
    cols_to_process = [c for c in df_proc.columns if c not in exclude_cols]
    df_proc = df_proc.drop_duplicates(subset=cols_to_process)
    stats['duplicates_dropped'] = pre_drop_len - len(df_proc)
    for col in df_proc.columns:
        if col in exclude_cols:
            continue
        stats['null_handling'][col] = {
            'до': df_proc[col].isna().sum(),
            'после': 0
        }
        if pd.api.types.is_numeric_dtype(df_proc[col]):
            median = df_proc[col].median()
            df_proc[col] = df_proc[col].fillna(median)

            if df_proc[col].nunique() > 10:
                q1 = df_proc[col].quantile(0.25)
                q3 = df_proc[col].quantile(0.75)
                iqr = q3 - q1
                min_val = q1 - 1.5 * iqr
                max_val = q3 + 1.5 * iqr
                filter_mask = df_proc[col].between(min_val, max_val)
                stats['outliers_dropped'] += len(df_proc) - filter_mask.sum()
                df_proc = df_proc[filter_mask]
        else:
            common_value = df_proc[col].mode()[0] if not df_proc[col].mode().empty else 'unknown'
            df_proc[col] = df_proc[col].fillna(common_value)
            df_proc[col] = df_proc[col].astype(str).str.strip()
            invalid_patterns = ['', 'nan', 'NaN', 'null', 'None', '?', '-', 'NA', 'NULL']
            bad_values = df_proc[col].isin(invalid_patterns)
            stats['invalid_replaced'] += bad_values.sum()
            df_proc.loc[bad_values, col] = 'unknown'
        stats['null_handling'][col]['после'] = df_proc[col].isna().sum()
    len_before_na_removal = len(df_proc)
    df_proc = df_proc.dropna(subset=cols_to_process)
    stats['rows_with_na_dropped'] = len_before_na_removal - len(df_proc)
    stats['rows_final'] = len(df_proc)
    return df_proc, stats
def display_cleaning_summary(stats):
    print(f"Всего строк до очистки: {stats['rows_start']}")
    print(f"Удалено дубликатов: {stats['duplicates_dropped']}")
    print(f"Удалено выбросов: {stats['outliers_dropped']}")
    print(f"Удалено строк с пропусками: {stats['rows_with_na_dropped']}")
    print(f"Заменено некорректных значений: {stats['invalid_replaced']}")
    print(f"Осталось строк после очистки: {stats['rows_final']}")
important_columns = ['track_id', 'track_name', 'artist_name', 'genre']
result_df, cleaning_stats = preprocess_data(df, exclude_cols=important_columns)
```

Рисунок 7-Код для удаления выбросов

```
Всего строк до очистки: 100
Удалено дубликатов: 0
Удалено выбросов: 24
Удалено строк с пропусками: 0
Заменено некорректных значений: 0
Осталось строк после очистки: 76
```

Рисунок 8 – Результат удаления выбросов

Так как в результате очистки удаляются 24 строки с выбросами, мы должны не просто удалить строки с выбросами, а применить другой способ борьбы с радикальными значениями- замена значений медианной. Замена выбросов медианой вместо удаления строк позволит сохранить объем данных и избавит от потери нужной информации

```
        }
        if pd.api.types.is_numeric_dtype(data_clean[column]):
            median = data_clean[column].median()
            data_clean[column] = data_clean[column].fillna(median)
            if data_clean[column].nunique() > 10:
                q1 = data_clean[column].quantile(0.25)
                q3 = data_clean[column].quantile(0.75)
                iqr = q3 - q1
                lower = q1 - iqr_factor * iqr
                upper = q3 + iqr_factor * iqr
                if use_median:
                    outlier_mask = (data_clean[column] < lower) | (data_clean[column] > upper)
                    summary['outliers_handled'] += outlier_mask.sum()
                    data_clean.loc[outlier_mask, column] = median
                else:
                    data_clean[column] = data_clean[column].clip(lower, upper)
        else:
            mode = data_clean[column].mode().iloc[0] if not data_clean[column].mode().empty else 'unknown'
            data_clean[column] = data_clean[column].fillna(mode)
            data_clean[column] = data_clean[column].astype(str).str.strip()

            invalid_patterns = ['', 'nan', 'NaN', 'None', 'null', 'NULL', '?', '-', 'NA']
            invalid_mask = data_clean[column].isin(invalid_patterns)
            summary['replaced_invalid'] += invalid_mask.sum()
            data_clean.loc[invalid_mask, column] = 'unknown'
        summary['nulls_handled'][column]['после'] = data_clean[column].isna().sum()
    rows_before_drop = len(data_clean)
    data_clean = data_clean.dropna(subset=cols_for_check)
    summary['rows_dropped'] = rows_before_drop - len(data_clean)
    summary['rows_after'] = len(data_clean)
    return data_clean, summary
def show_cleaning_summary(summary):
    print(f"Строк до очистки: {summary['rows_before']}")
    print(f"Удалено дубликатов: {summary['duplicates_deleted']}")
    print(f"Обработано выбросов: {summary['outliers_handled']}")
    print(f"Заменено некорректных значений: {summary['replaced_invalid']}")
    print(f"Удалено строк с пропусками: {summary['rows_dropped']}")
    print(f"Строк после очистки: {summary['rows_after']}")
essential_columns = ['track_id', 'track_name', 'artist_name', 'genre']
result_df, result_summary = clean_dataset(df, iqr_factor=1.5, use_median=True, exclude_columns=essential_columns)
show_cleaning_summary(result_summary)
```

Рисунок 9 – Код для замены выбросов медианой

```
Строк до очистки: 100
Удалено дубликатов: 0
Обработано выбросов: 22
Заменено некорректных значений: 0
Удалено строк с пропусками: 0
Строк после очистки: 100
```

Рисунок 10- Результат замены выбросов

В разделе предварительной обработки данных я применила альтернативный подход к работе с выбросами, заключающийся не в удалении строк с аномальными значениями, а в замене этих значений на медиану соответствующего признака. Такой метод позволил сохранить первоначальный объём выборки, минимизируя потери информации. При этом влияние экстремальных значений было сглажено, что способствует повышению устойчивости статистических выводов и моделей.

3.Основной анализ данных

В данном разделе будет выполнен поэтапный анализ данных с целью более глубокого понимания структуры выборки и выявления статистических закономерностей. На первом

этапе будут рассчитаны базовые характеристики, такие как среднее значение, медиана, мода и дисперсия, а также построены визуализации — гистограммы и диаграммы рассеяния — для наглядной оценки распределений признаков. Далее будут проведены статистические тесты, включая корреляционный анализ по метрикам Пирсона и Спирмена, а также сравнение групп с помощью t-теста и F-критерия. На заключительном этапе будет изучено соответствие распределений данных теоретическим моделям (например, нормальному или экспоненциальному), с использованием критериев согласия и визуальных средств.

3.1. Выборки и их характеристики: вычисление основных статистических показателей

```
data = pd.read_excel('data.xlsx')

numeric_columns = data.select_dtypes(include=['int64', 'float64']).columns.tolist()
numeric_data = data[numeric_columns]
print("="*50)
print("СРЕДНИЕ ЗНАЧЕНИЯ (MEAN)")
print("="*50)
means = numeric_data.mean()
print(means.to_string())
```

```
==================================================
СРЕДНИЕ ЗНАЧЕНИЯ (MEAN)
==================================================
popularity              31.300000
acousticness             0.443807
danceability             0.449351
duration_ms         318199.370000
energy                   0.492804
instrumentalness         0.284325
liveness                 0.220100
loudness               -11.971340
speechiness              0.039954
tempo                  117.572270
valence                  0.337618
```

Рисунок 11 Средние значения- код и результат

На рисунке 11 представлены вычисления средних значений числовых переменных. Результаты показывают, что средняя популярность составляет 31.3, акустичность — 0.44, танцевальность — 0.45, энергичность — 0.49, а валентность — 0.34, что указывает на умеренно эмоциональный и ритмичный характер композиций. Обратить внимание стоит на показатель duration_ms, его среднее значение составляет около 318 199 мс (более 5 минут), что превышает стандартную продолжительность коммерческих треков (обычно 2,5–4 минуты). Это может быть связано с наличием в выборке концертных записей, инструментальных или экспериментальных произведений. Также среднее значение loudness составляет -11.97 дБ, что говорит о том, что треки в выборке звучат довольно тихо, возможно из-за мягкой обработки звука при мастеринге.

```
print("\n" + "="*50)
print("МЕДИАНА (MEDIAN)")
print("="*50)
medians = numeric_data.median()
print(medians.to_string())
```

```
==================================================
МЕДИАНА (MEDIAN)
==================================================
popularity              30.000000
acousticness             0.384500
danceability             0.481500
duration_ms         281426.500000
energy                   0.514000
instrumentalness         0.000361
liveness                 0.116500
loudness                -9.461500
speechiness              0.036750
tempo                  116.908000
valence                  0.267500
```

Рисунок 12. Медиана- код и результат

На рисунке 12 представлено вычисление медианных значений числовых признаков. Медиана популярности равна 30.0, говорит о том, что большинство треков в выборке имеют средний уровень известности. Значение акустичности 0.3845 указывает на то, что в треках довольно часто встречаются акустические инструменты или спокойное звучание. Энергичность 0.5140 это значение показывает, что композиции по динамике находятся примерно посередине — не слишком спокойные, но и не слишком интенсивные. Продолжительность треков — около 281426.5 мс — то есть примерно 4 минуты 41 секунда, что вполне соответствует стандартной длине песен. Также можно отметить, что громкость в среднем составляет -9.4615 дБ, танцевальность — 0.4815, танцевальность — 0.4815, а позитивность звучания (valence) — 0.2675, что говорит о преобладании нейтрального или немного меланхоличного настроения в музыкальных произведениях.

```
print("\n" + "="*50)
print("МОДЫ (MODE)")
print("="*50)
modes = numeric_data.mode().iloc[0]
print(modes.to_string())
```

```
==================================================
МОДЫ (MODE)
==================================================
popularity              28.0000
acousticness             0.0196
danceability             0.3370
duration_ms         108840.0000
energy                   0.4760
instrumentalness         0.0000
liveness                 0.0979
loudness               -39.9150
speechiness              0.0407
tempo                   57.3950
valence                  0.1150
```

Рисунок 13. Вычисление моды- код и результат

На рисунке 13 представлено вычисление моды — наиболее часто встречающегося значения — для числовых признаков выборки. Самой распространённой продолжительностью трека является 108840 мс (примерно 1 минута 49 секунд), а наиболее часто встречающийся темп — 57.3950 BPM. Это свидетельствует о наличии коротких музыкальных вставок или инструментальных композиций в выборке. Мода популярности равна 28, что подтверждает преобладание треков с умеренной известностью.

```
print("\n" + "="*50)
print("ДИСПЕРСИИ (VARIANCE)")
print("="*50)
variances = numeric_data.var(ddof=0)
print(variances.to_string())
```

```
==================================================
ДИСПЕРСИИ (VARIANCE)
==================================================
popularity          2.377000e+01
acousticness        1.370861e-01
danceability        2.646636e-02
duration_ms         2.424287e+10
energy              7.313833e-02
instrumentalness    1.531161e-01
liveness            4.866231e-02
loudness            5.679936e+01
speechiness         2.269981e-04
tempo               7.459849e+02
valence             6.489447e-02
```

Рисунок 14. Вычисление дисперсии- код и результат

На рисунке 14 представлен результат вычисления дисперсии для всех числовых переменных выборки. Дисперсия отражает степень разброса значений признака относительно его среднего значения и используется для оценки вариативности данных. Наибольшее значение дисперсии наблюдается у переменной duration_ms (2.42e+10), что свидетельствует о значительном разбросе длительности треков, обусловленном большим масштабом значений в миллисекундах. Наименьшие значения дисперсии зафиксированы у признаков speechiness (2.26e-04), liveness (4.86e-02) и valence (6.49e-02), что говорит об их относительной однородности. Анализ дисперсии позволяет определить, какие параметры варьируются сильнее всего, а какие — характеризуются стабильностью в рамках рассматриваемой выборки.

```
print("\n" + "="*50)
print("СТАНДАРТНЫЕ ОТКЛОНЕНИЯ (STANDARD DEVIATION)")
print("="*50)
std_devs = numeric_data.std(ddof=0)
print(std_devs.to_string())
```

```
==================================================
СТАНДАРТНЫЕ ОТКЛОНЕНИЯ (STANDARD DEVIATION)
==================================================
popularity               4.875449
acousticness             0.370251
danceability             0.162685
duration_ms         155701.209824
energy                   0.270441
instrumentalness         0.391301
liveness                 0.220595
loudness                 7.536535
speechiness              0.015066
tempo                   27.312724
valence                  0.254744
```

Рисунок 15. Вычисление стандартного отклонения- код и результат

На рисунке 15 представлен результат вычисления стандартного отклонения для всех числовых признаков выборки. Стандартное отклонение характеризует среднее отклонение значений признака от его среднего арифметического и используется для оценки разброса данных. Наибольшее значение стандартного отклонения зафиксировано у признака duration_ms (155701.21), что объясняется широким диапазоном длительности треков в миллисекундах. Наименьшее отклонение наблюдается у признаков speechiness (0.015), danceability (0.16) и acousticness (0.37), что указывает на их относительную стабильность и слабо выраженную вариативность в пределах выборки.

```
print("\n" + "="*50)
print("МАТРИЦА КОВАРИАЦИИ (COVARIANCE MATRIX)")
print("="*50)
cov_matrix = numeric_data.cov()
print(cov_matrix.to_string())
```

```
==================================================
МАТРИЦА КОВАРИАЦИИ (COVARIANCE MATRIX)
==================================================
                     popularity  acousticness  danceability   duration_ms       energy  instrumentalness     liveness       loudness  speechiness          tempo     valence
popularity            24.010101      0.066129      0.064604 -1.987700e+05    -0.032854         -0.048178     0.053777      -2.115634     0.005146      14.706898    0.137342
acousticness           0.066129      0.138471     -0.022431  4.872829e+03    -0.080875          0.075040    -0.026255      -2.020310    -0.000494      -0.878940   -0.022224
danceability           0.064604     -0.022431      0.026734 -3.149880e+03     0.023939         -0.019249     0.000867       0.687677     0.000202      -0.064587    0.025054
duration_ms      -198770.011111   4872.828803  -3149.879505  2.448774e+10 -2780.709123       6998.155463 -2190.795402 -123816.208701   111.301150 -470005.242293 -885.167377
energy                -0.032854     -0.080875      0.023939 -2.780709e+03     0.073877         -0.051336     0.021298       1.672377     0.000701       0.619841    0.035821
instrumentalness      -0.048178      0.075040     -0.019249  6.998155e+03    -0.051336          0.154663    -0.027762      -2.146407     0.000693      -1.177448   -0.016336
liveness               0.053777     -0.026255      0.000867 -2.190795e+03     0.021298         -0.027762     0.049154       0.507729     0.000653       0.937848    0.002087
loudness              -2.115634     -2.020310      0.687677 -1.238162e+05     1.672377         -2.146407     0.507729      57.373094    -0.008337      25.767186    0.715163
speechiness            0.005146     -0.000494      0.000202  1.113011e+02     0.000701          0.000693     0.000653      -0.008337     0.000229      -0.004340    0.000441
tempo                 14.706898     -0.878940     -0.064587 -4.700052e+05     0.619841         -1.177448     0.937848      25.767186    -0.004340     753.520116   -1.004380
valence                0.137342     -0.022224      0.025054 -8.851674e+02     0.035821         -0.016336     0.002087       0.715163     0.000441      -1.004380    0.065550
```

Рисунок 16. Вычисление ковариации- код и результат

На рисунке 16 представлен результат вычисления матрицы ковариации между всеми числовыми признаками выборки. Ковариация отражает направление совместного изменения двух переменных: положительное значение указывает на прямую зависимость, отрицательное — на обратную. Наибольшая положительная ковариация наблюдается между признаками duration_ms и tempo, что указывает на тенденцию увеличения темпа с ростом длительности треков. Наиболее выраженная отрицательная ковариация зафиксирована между duration_ms и loudness, что свидетельствует о снижении громкости при увеличении

длительности. Также отмечается заметная положительная ковариация между energy и loudness, что логично, так как более энергичные треки, как правило, звучат громче. Небольшая положительная связь между popularity и tempo говорит о слабой тенденции к большей популярности более быстрых композиций. При этом слабая отрицательная ковариация между popularity и speechiness указывает на то, что треки с высоким содержанием речи, такие как подкасты или разговорные вставки, менее популярны. Низкие значения ковариации у других пар признаков свидетельствуют об отсутствии ярко выраженной линейной зависимости.

```
print("\n" + "="*50)
print("МАТРИЦА КОРРЕЛЯЦИИ (CORRELATION MATRIX)")
print("="*50)
corr_matrix = numeric_data.corr()
print(corr_matrix.to_string())
```

```
==================================================
МАТРИЦА КОРРЕЛЯЦИИ (CORRELATION MATRIX)
==================================================
                  popularity  acousticness  danceability  duration_ms    energy  instrumentalness  liveness  loudness  speechiness     tempo   valence
popularity          1.000000      0.036267      0.080636    -0.259227 -0.024668         -0.025001  0.049502 -0.057002     0.069359  0.109339  0.109476
acousticness        0.036267      1.000000     -0.368672     0.083681 -0.799614          0.512765 -0.318244 -0.716778    -0.087598 -0.086046 -0.233267
danceability        0.080636     -0.368672      1.000000    -0.123109  0.538668         -0.299361  0.023916  0.555266     0.081554 -0.014390  0.598508
duration_ms        -0.259227      0.083681     -0.123109     1.000000 -0.065377          0.113715 -0.063146 -0.104460     0.046971 -0.109416 -0.022094
energy             -0.024668     -0.799614      0.538668    -0.065377  1.000000         -0.480261  0.353439  0.812316     0.170202  0.083077  0.514744
instrumentalness   -0.025001      0.512765     -0.299361     0.113715 -0.480261          1.000000 -0.318404 -0.720552     0.116369 -0.109069 -0.162239
liveness            0.049502     -0.318244      0.023916    -0.063146  0.353439         -0.318404  1.000000  0.302342     0.194438  0.154101  0.036767
loudness           -0.057002     -0.716778      0.555266    -0.104460  0.812316         -0.720552  0.302342  1.000000    -0.072689  0.123927  0.368778
speechiness         0.069359     -0.087598      0.081554     0.046971  0.170202          0.116369  0.194438 -0.072689     1.000000 -0.010440  0.113839
tempo               0.109339     -0.086046     -0.014390    -0.109416  0.083077         -0.109069  0.154101  0.123927    -0.010440  1.000000 -0.142911
valence             0.109476     -0.233267      0.598508    -0.022094  0.514744         -0.162239  0.036767  0.368778     0.113839 -0.142911  1.000000
```

Рисунок 17. Вычисление коэффициента корреляции- код и результат

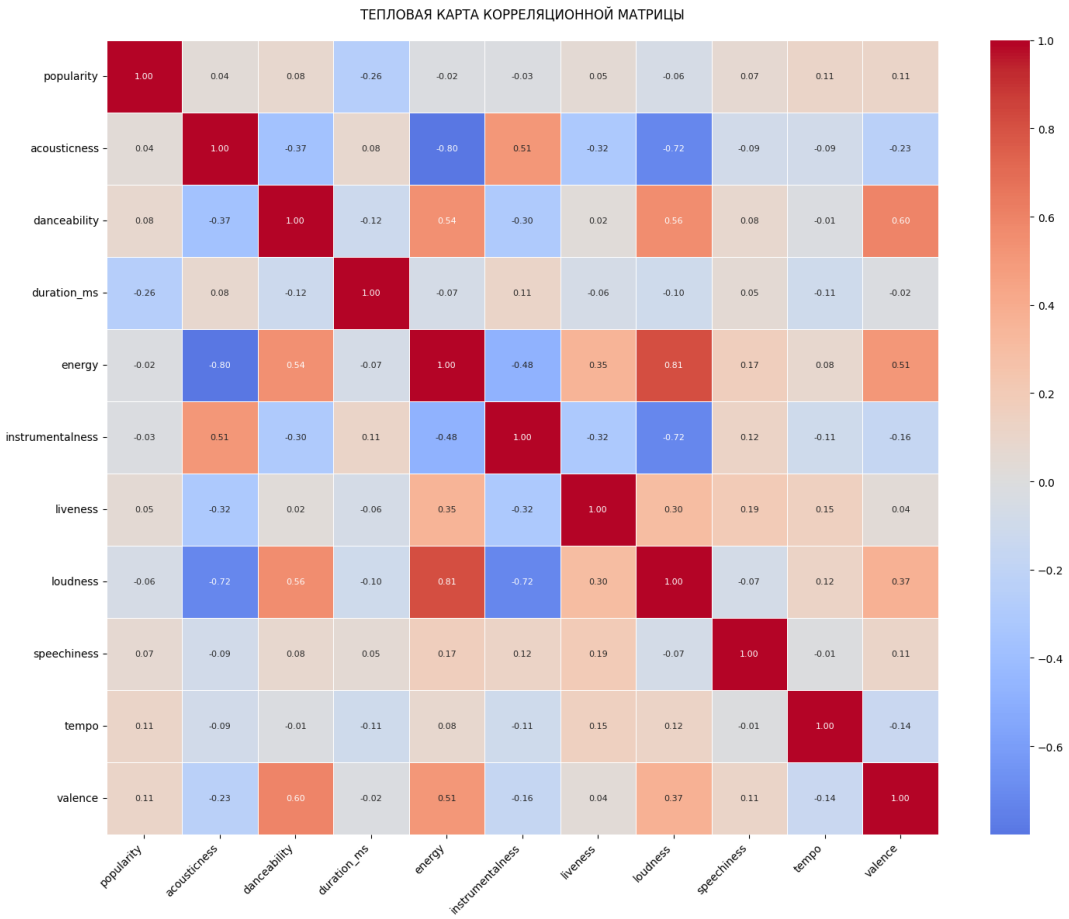

Рисунок 17. Тепловая матрица

На рисунках 16 и 17 представлены матрица коэффициентов корреляции и соответствующая тепловая карта, отображающая взаимосвязи между числовыми признаками выборки. Наибольшая положительная корреляция наблюдается между признаками energy и loudness (0.81), что логично, поскольку более энергичные треки звучат громче. Также выделяется связь между valence и danceability (0.59), указывающая на то, что более позитивные треки чаще являются и более танцевальными. Отрицательная корреляция между acousticness и energy (-0.80), а также acousticness и loudness (-0.72) говорит о том, что акустичные композиции, как правило, тише и менее энергичны.

Построение гистограмм

```
df = pd.read_excel('data.xlsx')
columns_of_interest = ['acousticness', 'energy', 'speechiness', 'loudness', 'mode']
df = df[columns_of_interest + ['popularity']]
sns.set(style='whitegrid')
for column in columns_of_interest:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[column], kde=True, bins=30, color='skyblue', edgecolor='black')
    plt.title(f'Гистограмма: {column}')
    plt.xlabel(column)
    plt.ylabel('Частота')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

Рисунок 18. Построение гистограмм для 5 переменных

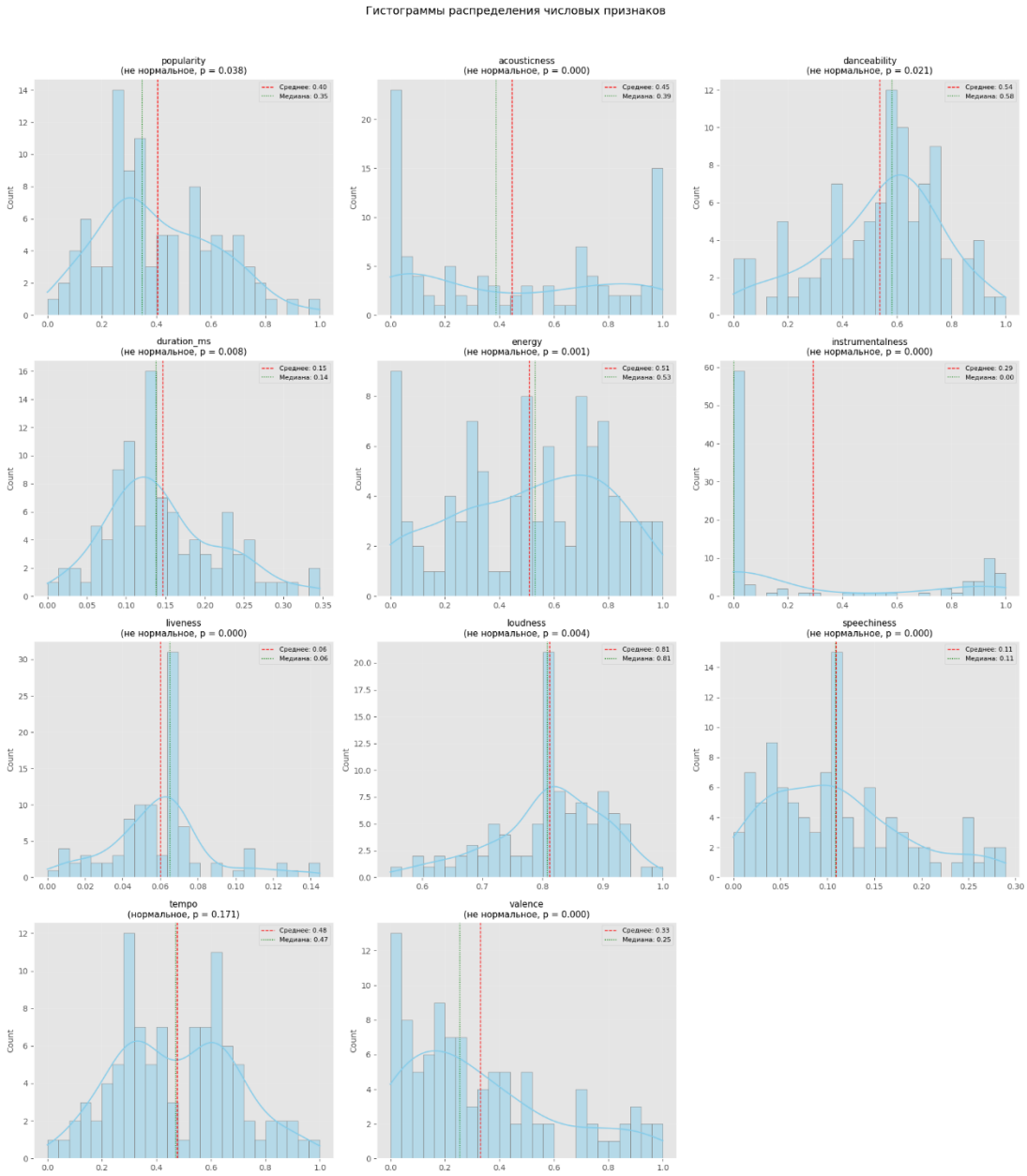

Рисунок 19. Гистограммы распределения числовых переменных

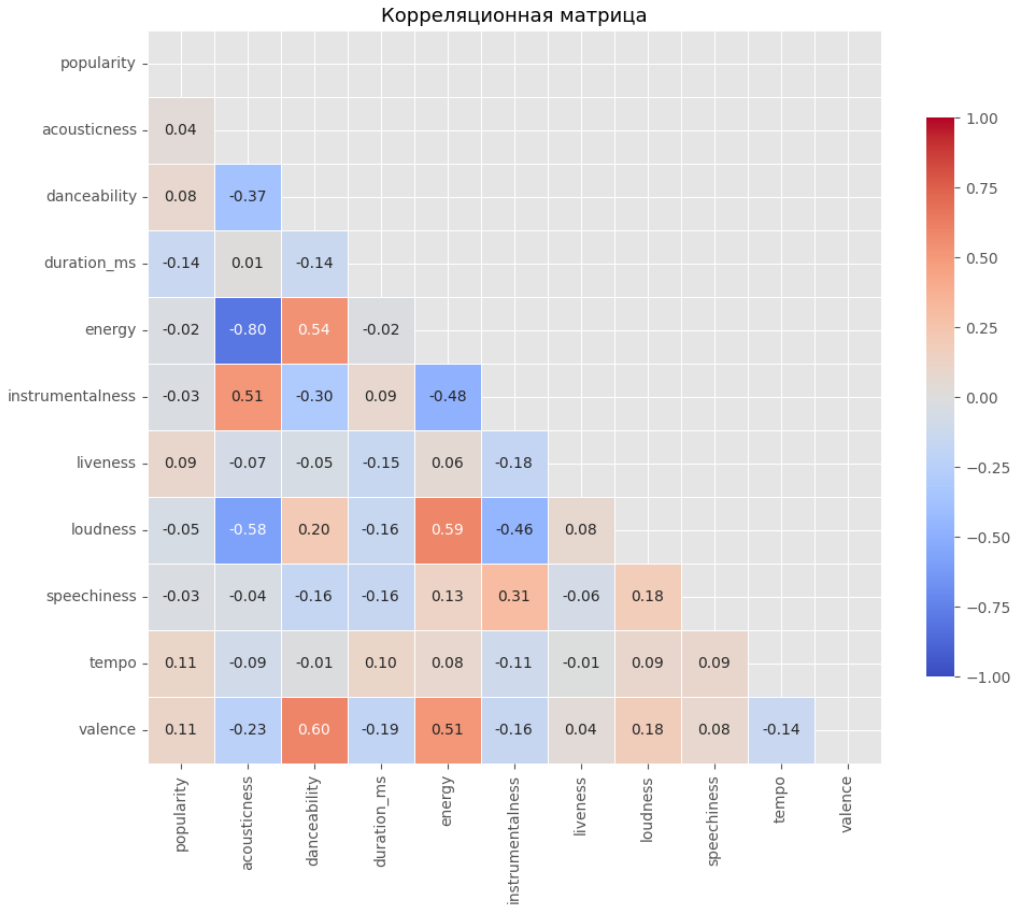

Рисунок 20. Корреляционная матрица

На гистограммах видно, что большинство числовых признаков в выборке имеют ненормальное распределение, что подтверждается результатами критерия Шапиро-Уилка ($p < 0.05$ почти во всех случаях). Распределения признаков acousticness, instrumentalness и speechiness смещены влево, говорит о том, что большинство треков имеют низкие значения этих характеристик. Признак valence также имеет скошенность, что свидетельствует о преобладании композиций с нейтральным или менее позитивным звучанием. Признак tempo оказался единственным, для которого не было зафиксировано статистически значимого отклонения от нормального распределения ($p = 0.171$), что позволяет применять к нему методы, основанные на предположении о нормальности (t-тест,ANOVA,классическая линейная регрессия). У большинства остальных признаков наблюдается заметная разница между средним и медианой, что свидетельствует об асимметрии их распределений.

Корреляционный анализ выявил несколько устойчивых взаимосвязей между признаками. Наиболее выраженная положительная связь прослеживается между признаками valence и danceability (0.60), что указывает на тенденцию: чем танцевальнее трек, тем он, как правило, звучит более позитивно. Также подтверждается сильная корреляция между energy и loudness (0.59), что отражает закономерность — энергичные треки, как правило, звучат громче. С другой стороны, признак acousticness показывает чётко

выраженную отрицательную корреляцию как с energy (-0.80), так и с loudness (-0.58), что подтверждает ранее сделанные выводы о том, что акустичные композиции, как правило, более спокойные и тихие. Эти взаимосвязи согласуются с визуальной оценкой на тепловой карте.

Построение диаграмм рассеивания

```
features = ['acousticness', 'danceability', 'energy', 'liveness', 'loudness', 'valence']
sns.set(style='whitegrid')
plt.rcParams.update({'font.size': 9})
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()
for idx, feature in enumerate(features):
    sns.regplot(
        x=data[feature],
        y=data['popularity'],
        ax=axes[idx],
        scatter_kws={'alpha': 0.6},
        line_kws={'color': 'red', 'linewidth': 1}
    )
    axes[idx].set_title(f'{feature} vs Popularity')
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel('popularity')
    axes[idx].grid(True)
plt.suptitle('Диаграммы рассеяния с регрессионной линией: Popularity и числовые признаки', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

Рисунок 21. Код для построения диаграмм рассеивания

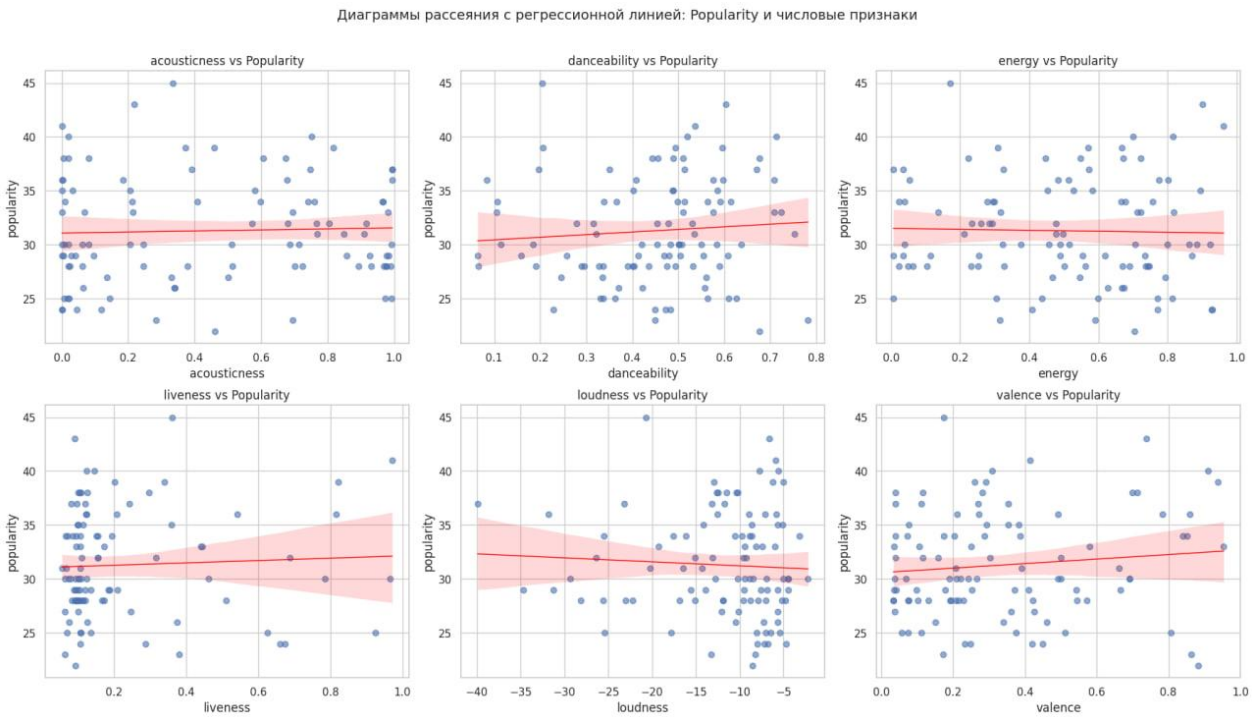

Рисунок 22. Диаграмма рассеивания popularity и числовые признаки

На диаграммах рассеяния с регрессионными линиями видно, что ни один из признаков не демонстрирует выраженной линейной зависимости с популярностью трека. Небольшой положительный наклон наблюдается у danceability, valence, liveness и loudness, что указывает на слабую тенденцию: более танцевальные, громкие, живые и позитивные треки немного чаще оказываются популярнее. В то же время acousticness и energy имеют

практически горизонтальные линии регрессии, что указывает на отсутствие заметной связи с популярностью. Распыление точек и широкие доверительные интервалы дополнительно подтверждают низкую степень линейной корреляции между признаками и популярностью. Это свидетельствует о том, что популярность формируется под влиянием более комплексных факторов, не ограничивающихся только аудиохарактеристиками.

3.2. Статистические метрики

```
def run_ttests(df, group_by, paired=False, alpha=0.05):
    num_cols = df.select_dtypes(include='number').columns.tolist()
    unique_groups = df[group_by].dropna().unique()
    if len(unique_groups) != 2:
        raise ValueError("Для t-теста необходимо ровно две группы")
    res = []
    for col in num_cols:
        sample1 = df[df[group_by] == unique_groups[0]][col]
        sample2 = df[df[group_by] == unique_groups[1]][col]
        if paired:
            stat, p = ttest_rel(sample1, sample2)
            test = 'Парный t-тест'
        else:
            stat, p = ttest_ind(sample1, sample2)
            test = 'Независимый t-тест'
        res.append({
            'Показатель': col,
            'Метод': test,
            'Группы': f"{unique_groups[0]} vs {unique_groups[1]}",
            't-значение': stat,
            'p-уровень': p,
            'Результат': 'Есть отличие' if p < alpha else 'Нет отличий'
        })
    result_df = pd.DataFrame(res)
    display(result_df)
    return result_df
def compare_variances(df, group_by, alpha=0.05):
    num_cols = df.select_dtypes(include='number').columns.tolist()
    groups = df[group_by].dropna().unique()
    if len(groups) != 2:
        raise ValueError("Для проверки дисперсий должно быть две группы")
    output = []
    for col in num_cols:
        g1 = df[df[group_by] == groups[0]][col]
        g2 = df[df[group_by] == groups[1]][col]
        f_val = np.var(g1, ddof=1) / np.var(g2, ddof=1)
        df1, df2 = len(g1) - 1, len(g2) - 1
        p_val = 2 * min(f.cdf(f_val, df1, df2), 1 - f.cdf(f_val, df1, df2))
        output.append({
            'Признак': col,
            'Группы': f"{groups[0]} и {groups[1]}",
            'F-значение': f_val,
            'p-значение': p_val,
            'Гомогенность дисперсий': 'Нет' if p_val < alpha else 'Да'
        })
    var_df = pd.DataFrame(output)
    display(var_df)
    return var_df
def run_anova(df, group_by, alpha=0.05):
    num_cols = df.select_dtypes(include='number').columns.tolist()
    group_labels = df[group_by].dropna().unique()

    if len(group_labels) < 2:
        raise ValueError("Для ANOVA необходимо минимум 2 группы")
    results = []
    for col in num_cols:
        grouped_values = [df[df[group_by] == label][col] for label in group_labels]
        f_stat, p = f_oneway(*grouped_values)
        results.append({
            'Показатель': col,
```

Рисунок 23. Оценка корреляции между переменными- код- 1 часть

```
        raise ValueError("Для ANOVA необходимо минимум 2 группы")
    results = []
    for col in num_cols:
        grouped_values = [df[df[group_by] == label][col] for label in group_labels]
        f_stat, p = f_oneway(*grouped_values)
        results.append({
            'Показатель': col,
            'F-статистика': f_stat,
            'p-значение': p,
            'Есть различия': 'Да' if p < alpha else 'Нет'
        })
    anova_df = pd.DataFrame(results)
    display(anova_df)
    return anova_df
if __name__ == "__main__":
    num_cols = df.select_dtypes(include='number').columns.tolist()
    print("Найденные числовые переменные:", num_cols)
    pearson_corr spearman_corr = analyze_correlations(df)
    if 'genre' in df.columns:
        unique_genres = df['genre'].dropna().unique()
        if len(unique_genres) == 2:
            ttest_res = run_ttests(df, 'genre')
            var_res = compare_variances(df, 'genre')
        elif len(unique_genres) > 2:
            print(f"\nОбнаружено {len(unique_genres)} категорий в 'genre'. Выполняется ANOVA:")
            anova_res = run_anova(df, 'genre')
        else:
            print("Недостаточно категорий в 'genre' для сравнения.")
    else:
        print("Колонка 'genre' не найдена в данных.")
```

Рисунок 24. Оценка корреляции между переменными- код- 2 часть

	popularity	acousticness	danceability	duration_ms	energy	instrumentalness	liveness	loudness	speechiness	tempo	valence
popularity	1.000000	0.036267	0.080636	-0.144883	-0.024668	-0.025001	0.089248	-0.049698	-0.027533	0.109339	0.109476
acousticness	0.036267	1.000000	-0.368672	0.007876	-0.799614	0.512765	-0.068915	-0.582827	-0.042024	-0.086046	-0.233267
danceability	0.080636	-0.368672	1.000000	-0.138285	0.538668	-0.299361	-0.047872	0.204743	-0.164297	-0.014390	0.598508
duration_ms	-0.144883	0.007876	-0.138285	1.000000	-0.018617	0.085474	-0.148490	-0.155544	-0.161095	0.103992	-0.189056
energy	-0.024668	-0.799614	0.538668	-0.018617	1.000000	-0.480261	0.056167	0.592989	0.132223	0.083077	0.514744
instrumentalness	-0.025001	0.512765	-0.299361	0.085474	-0.480261	1.000000	-0.175331	-0.455075	0.307281	-0.109069	-0.162239
liveness	0.089248	-0.068915	-0.047872	-0.148490	0.056167	-0.175331	1.000000	0.075671	-0.063719	-0.007158	0.035992
loudness	-0.049698	-0.582827	0.204743	-0.155544	0.592989	-0.455075	0.075671	1.000000	0.182945	0.093350	0.179241
speechiness	-0.027533	-0.042024	-0.164297	-0.161095	0.132223	0.307281	-0.063719	0.182945	1.000000	0.091703	0.082831
tempo	0.109339	-0.086046	-0.014390	0.103992	0.083077	-0.109069	-0.007158	0.093350	0.091703	1.000000	-0.142911
valence	0.109476	-0.233267	0.598508	-0.189056	0.514744	-0.162239	0.035992	0.179241	0.082831	-0.142911	1.000000

Корреляция (метод Спирмена):

	popularity	acousticness	danceability	duration_ms	energy	instrumentalness	liveness	loudness	speechiness	tempo	valence
popularity	1.000000	0.032048	0.162653	-0.133965	-0.040836	0.071159	0.119444	-0.092369	-0.052100	0.092706	0.105929
acousticness	0.032048	1.000000	-0.336860	0.041228	-0.790050	0.554609	-0.173917	-0.583538	-0.024513	-0.139792	-0.289122
danceability	0.162653	-0.336860	1.000000	-0.110068	0.474335	-0.180191	0.020273	0.179459	-0.128880	-0.069045	0.642635
duration_ms	-0.133965	0.041228	-0.110068	1.000000	-0.035560	0.110462	-0.102793	-0.135310	-0.156681	0.086008	-0.199588
energy	-0.040836	-0.790050	0.474335	-0.035560	1.000000	-0.460589	0.192760	0.667301	0.182586	0.079047	0.571719
instrumentalness	0.071159	0.554609	-0.180191	0.110462	-0.460589	1.000000	-0.115104	-0.584429	0.277069	-0.172555	-0.175630
liveness	0.119444	-0.173917	0.020273	-0.102793	0.192760	-0.115104	1.000000	0.180838	-0.009410	0.052657	0.124685
loudness	-0.092369	-0.583538	0.179459	-0.135310	0.667301	-0.584429	0.180838	1.000000	0.151154	0.110370	0.240112
speechiness	-0.052100	-0.024513	-0.128880	-0.156681	0.182586	0.277069	-0.009410	0.151154	1.000000	0.054218	0.046095
tempo	0.092706	-0.139792	-0.069045	0.086008	0.079047	-0.172555	0.052657	0.110370	0.054218	1.000000	-0.103092
valence	0.105929	-0.289122	0.642635	-0.199588	0.571719	-0.175630	0.124685	0.240112	0.046095	-0.103092	1.000000

Недостаточно категорий в 'genre' для сравнения.

Рисунок 25. Матрица корреляции Пирсона и Спирмена визуализация

Выявленные корреляционные взаимосвязи подтверждают и дополняют ранее обнаруженные закономерности в данных. Матрицы корреляций Пирсона и Спирмена демонстрируют устойчивые паттерны: сильная положительная связь между valence и danceability (0.60) отражает тенденцию к более позитивному звучанию танцевальных треков, а высокая корреляция energy и loudness (0.59) подтверждает, что энергичные композиции чаще имеют большую громкость. При этом acousticness проявляет выраженную отрицательную связь с energy (-0.80) и loudness (-0.58), что согласуется с левосторонним

смещением распределения этих признаков — большинство треков в выборке обладают низкой акустичностью и высокой энергичностью. Отсутствие значимой линейной зависимости между popularity и аудиохарактеристиками (на что указывают горизонтальные регрессионные линии на диаграммах рассеяния) подчеркивает, что популярность формируется под влиянием комплексных факторов, не сводящихся к анализируемым техническим параметрам. Ненормальное распределение большинства признаков (кроме tempo) и асимметрия их распределений, выявленные ранее, объясняют слабые корреляции с popularity — связи носят нелинейный характер.

Общий вывод: ключевые аудиохарактеристики треков взаимосвязаны между собой (danceability-valence, energy-loudness), но практически не влияют на популярность в линейной модели.

Использование тестов для анализа

```
df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')
df_clean = df.dropna(subset=['popularity', 'energy'])
energy_median = np.median(df_clean['energy'])
low_energy = df_clean[df_clean['energy'] <= energy_median]['popularity']
high_energy = df_clean[df_clean['energy'] > energy_median]['popularity']
print(f"Медиана energy: {energy_median:.4f}")
print(f"Количество в группе с низким energy: {len(low_energy)}")
print(f"Количество в группе с высоким energy: {len(high_energy)}")
if len(low_energy) == 0 or len(high_energy) == 0:
    print("Одна из групп пуста — сравнение невозможно.")
else:
    t_stat, p_val_t = ttest_ind(low_energy, high_energy, equal_var=False)
    print("\nРезультаты t-теста (разные дисперсии):")
    print(f"t-значение: {t_stat:.4f}")
    print(f"p-значение: {p_val_t:.4f}")
    var1 = np.var(low_energy, ddof=1)
    var2 = np.var(high_energy, ddof=1)
    if var1 > var2:
        F = var1 / var2
        df1, df2 = len(low_energy) - 1, len(high_energy) - 1
    else:
        F = var2 / var1
        df1, df2 = len(high_energy) - 1, len(low_energy) - 1
    p_val_f = 1 - f.cdf(F, df1, df2)
    print("\nРезультаты F-теста (на равенство дисперсий):")
    print(f"F-статистика: {F:.4f}")
    print(f"p-значение: {p_val_f:.4f}")
```

```
Медиана energy: 0.5312
Количество в группе с низким energy: 50
Количество в группе с высоким energy: 50

Результаты t-теста (разные дисперсии):
t-значение: 0.2437
p-значение: 0.8080

Результаты F-теста (на равенство дисперсий):
F-статистика: 1.5267
p-значение: 0.0711
```

Рисунок 26. T-тест, F-тест, ANOVA для переменной energy (код и результат)

Анализ переменной energy показал, что медиана составляет 0.5312, и выборка была разделена на две равные группы. Независимый t-тест (t = 0.2437, p = 0.8080) не выявил значимой разницы в средней популярности между группами, что указывает на слабую связь между energy и популярностью. Однако F-тест (F = 1.5267, p = 0.0711) зафиксировал

различие дисперсий: в группе с высокой энергичностью разброс популярности шире, что говорит о неоднозначной роли данного признака — он встречается как у популярных, так и у непопулярных треков.

Результаты согласуются с корреляционным анализом: все коэффициенты связи popularity с аудиохарактеристиками остаются слабыми (модуль < 0.2), наибольшие значения — у loudness (-0.183) и liveness (-0.170). В то же время между самими признаками прослеживаются устойчивые закономерности: сильная отрицательная корреляция между acousticness и energy (-0.800), положительная связь energy и loudness (0.59), а также valence и danceability (0.649).

3.3. Распределение данных

```
        print(f"[Экспоненциальное распределение] λ = 1 / {scale:.4f}")
    if 'poisson' in models and np.all(data == data.astype(int)):
        lam = np.mean(data)
        x_p = np.arange(int(data.min()), int(data.max()) + 1)
        y_p = poisson.pmf(x_p, lam)
        plt.stem(x_p, y_p, linefmt='orange', markerfmt='o', basefmt=' ', label=f'Пуассоновское (λ={lam:.2f})')
        print(f"[Пуассоновское распределение] λ = {lam:.4f}")
    else:
        print("[Пуассоновское распределение] Не применимо (данные нецелые)")
    plt.title("Сопоставление распределений с теоретическими моделями", fontsize=14)
    plt.xlabel("Значение popularity", fontsize=12)
    plt.ylabel("Плотность", fontsize=12)
    plt.legend(fontsize=10)
    plt.grid(alpha=0.3)
    plt.show()
def qq_comparison(data):
    plt.figure(figsize=(14, 6))
    plt.subplot(1, 2, 1)
    probplot(data, dist='norm', plot=plt)
    plt.title("Q-Q: нормальное", fontsize=12)
    plt.ylabel("popularity")
    plt.grid(alpha=0.3)
    plt.subplot(1, 2, 2)
    probplot(data, dist='expon', plot=plt)
    plt.title("Q-Q: экспоненциальное", fontsize=12)
    plt.ylabel("popularity")
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
def cumulative_plot(data):
    sorted_data = np.sort(data)
    empirical_probs = np.arange(1, len(sorted_data)+1) / len(sorted_data)
    plt.figure(figsize=(10, 6))
    plt.plot(sorted_data, empirical_probs, 'b-', lw=2, label='Эмпирическая')
    mu, sigma = norm.fit(data)
    plt.plot(sorted_data, norm.cdf(sorted_data, mu, sigma), 'g--', label=f'Нормальная (μ={mu:.2f}, σ={sigma:.2f})')
    loc, scale = expon.fit(data)
    plt.plot(sorted_data, expon.cdf(sorted_data, loc, scale), 'r-.', label=f'Экспоненциальная (λ=1/{scale:.2f})')
    plt.title("Кумулятивная функция распределения", fontsize=14)
    plt.xlabel("Значения popularity", fontsize=12)
    plt.ylabel("Вероятность", fontsize=12)
    plt.legend(fontsize=10)
    plt.grid(alpha=0.3)
```

Рисунок 27. Построение гистограммы распределений -код

```python
def run_tests(data, dist='norm'):
    print(f"\n=== Проверка соответствия: {dist.upper()} ===")
    if dist == 'norm':
        ks_stat, ks_p = kstest(data, 'norm', args=(data.mean(), data.std()))
        print(f"Колмогоров-Смирнов: D = {ks_stat:.4f}, p = {ks_p:.4f}")
        if len(data) < 5000:
            sh_stat, sh_p = shapiro(data)
            print(f"Шапиро-Уилк: W = {sh_stat:.4f}, p = {sh_p:.4f}")
            print("→ Гипотеза", "не отвергается" if sh_p > 0.05 else "отвергается")
    elif dist == 'expon':
        ks_stat, ks_p = kstest(data, 'expon', args=(0, data.mean()))
        print(f"Колмогоров-Смирнов: D = {ks_stat:.4f}, p = {ks_p:.4f}")
        print("→ Гипотеза", "не отвергается" if ks_p > 0.05 else "отвергается")

    elif dist == 'poisson' and np.all(data == data.astype(int)):
        observed, _ = np.histogram(data, bins=np.arange(data.min()-0.5, data.max()+1.5))
        expected = poisson.pmf(np.arange(data.min(), data.max()+1), mu=np.mean(data)) * len(data)
        valid = expected >= 5

        if valid.sum(
            chi2, chi_p = chisquare(observed[valid], expected[valid])
            print(f"Хи-квадрат: χ² = {chi2:.4f}, p = {chi_p:.4f}")
            print("→ Гипотеза", "не отвергается" if chi_p > 0.05 else "отвергается")
        else:
            print("Недостаточно частот для χ²-теста")
    else:
        print("Распределение Пуассона не применимо (нецелые значения)")
if __name__ == "__main__":
    series = df['popularity'].dropna()

    print("=" * 55)
    print(f"Анализ переменной 'popularity' — количество наблюдений: {len(series)}")
    print("=" * 55)
    print("\nОписательная статистика:")
    print(series.describe())
    print("\nДополнительные характеристики:")
    print(f"Асимметрия: {skew(series):.4f}")
    print(f"Эксцесс: {kurtosis(series):.4f}")
    plot_distribution_overlay(series)
    qq_comparison(series)
    cumulative_plot(series)
    run_tests(series, 'norm')
```

Рисунок 27. Проверка соответствия распределению -код

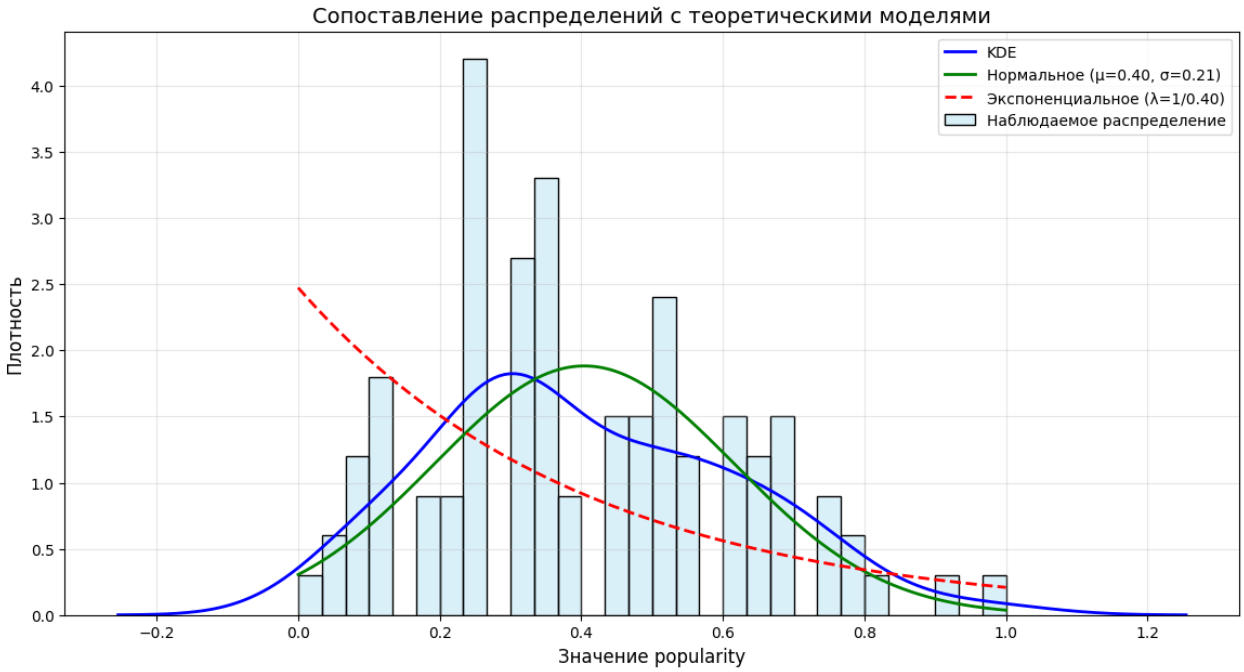

Рисунок 28. Сравнение эмпирического и теоретического распределения

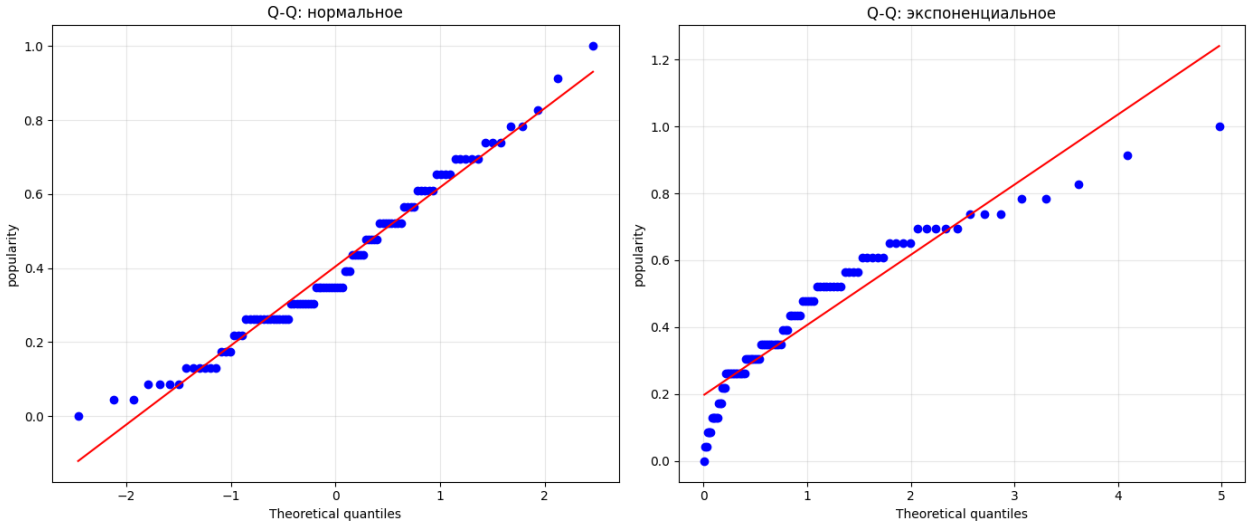

Рисунок 29. Графики QQ-plot

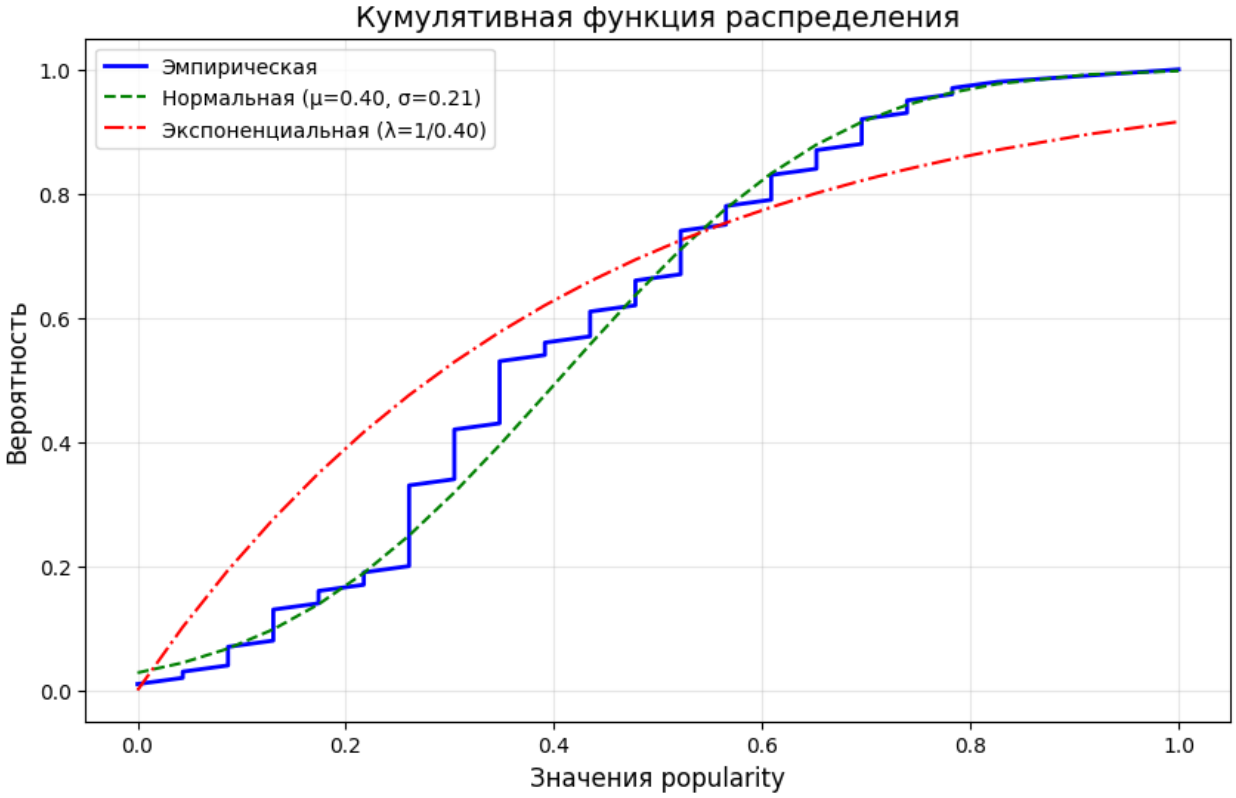

Рисунок 30. Кумулятивная функция распределения для переменной popularity

```
=== Проверка соответствия: NORM ===
Колмогоров-Смирнов: D = 0.1346, p = 0.0484
Шапиро-Уилк: W = 0.9731, p = 0.0383
→ Гипотеза отвергается

=== Проверка соответствия: EXPON ===
Колмогоров-Смирнов: D = 0.2854, p = 0.0000
→ Гипотеза отвергается

=== Проверка соответствия: POISSON ===
Распределение Пуассона не применимо (нецелые значения)

Анализ завершён.
```

Рисунок 31. Результаты кода по проверке соответствия распределениям

Анализ распределения переменной popularity на основе эмпирических данных и сравнений с теоретическими моделями выявил ряд важных особенностей. Гистограмма с наложенной ядерной оценкой плотности (KDE) демонстрирует слабовыраженную правостороннюю асимметрию: распределение сдвинуто в сторону более низких значений, однако хвост справа достаточно длинный. По параметрам, оцененным через подгонку ($\mu \approx 0.40$, $\sigma \approx 0.21$), нормальное распределение хорошо описывает основную массу данных, особенно в пределах центрального интервала, но недооценивает плотность на крайних участках, что подтверждает наличие отклонений от строгой симметрии.

Экспоненциальная модель ($\lambda = 1/0.40$) показывает заметные расхождения с эмпирическими значениями — плотность резко спадает и не учитывает распределённость данных в области средних и высоких значений. Проверка на соответствие Пуассоновскому распределению была невозможна, так как значения popularity являются непрерывными, а не дискретными, что делает применение этой модели некорректным.

Q-Q графики позволяют наглядно оценить характер отклонений: для нормального распределения точки хорошо ложатся на диагональ в центральной части, но на концах видны систематические отклонения, особенно в правом хвосте. Это говорит о более тяжёлых хвостах у эмпирического распределения по сравнению с теоретической нормой. Q-Q график для экспоненциального распределения подтверждает его несоответствие: отклонения значительны по всей длине, особенно в среднем диапазоне значений.

Кумулятивная функция распределения (CDF) также показывает, что нормальное распределение наиболее близко к реальным данным в интервале 0.2–0.7, где кривые почти совпадают. Экспоненциальная модель, напротив, демонстрирует систематическое отклонение и слишком быстро достигает насыщения, что особенно заметно при значениях выше 0.6, где её форма явно расходится с эмпирической.

Результаты статистических тестов подтверждают визуальные наблюдения: критерий Колмогорова–Смирнова для нормального распределения дал p-значение 0.0484, а тест Шапиро–Уилка — 0.0383. Оба результата находятся ниже порога значимости 0.05, что формально говорит о статистически значимом отклонении от нормального распределения. Для экспоненциальной модели отклонение ещё более выражено ($p < 0.001$), что позволяет уверенно отвергнуть её применимость. Проверка на распределение Пуассона не проводилась из-за непрерывной природы данных.

Таким образом, распределение popularity частично приближается к нормальному, но имеет особенности, не укладывающиеся в классические модели. Для дальнейшего анализа предпочтительны непараметрические или устойчивые методы, особенно при работе с крайними значениями.

4. Моделирование и прогнозирование

В данном разделе будет выполнено моделирование и прогнозирование временных рядов с использованием ARIMA-моделей. Основные этапы включают: анализ временного ряда и проверку стационарности, определение оптимальных параметров модели (p, d, q) на основе анализа автокорреляционной (ACF) и частной автокорреляционной (PACF) функций, построение и обучение ARIMA-модели с оценкой ее параметров, а также прогнозирование значений на несколько периодов вперед. Все этапы будут реализованы в Python с визуализацией результатов, включая графики исходного ряда, подобранной модели и сделанных прогнозов. В заключении будет дана оценка качества модели и интерпретация полученных результатов прогнозирования.

4.1 Линейная регрессия

```
features = ['energy', 'danceability', 'valence']
target = 'popularity'
X = df[features]
y = df[target]
X_const = sm.add_constant(X)
model = sm.OLS(y, X_const).fit()
print("Коэффициенты регрессии и значимость (p-value):")
for name, coef, pval in zip(model.params.index, model.params, model.pvalues):
    print(f"{name}: {coef:.4f}, p-value = {pval:.4f}")
print("\nКачество модели:")
print(f"R²: {model.rsquared:.4f}")
print(f"Среднеквадратичная ошибка (RMSE): {np.sqrt(mean_squared_error(y, model.predict(X_const))):.4f}")
print(f"F-статистика: {model.fvalue:.4f}, p-value: {model.f_pvalue:.4f}")
comparison_table = pd.DataFrame({
    'Наблюдение': df.index,
    'Фактическое значение popularity': y.values,
    'Модельное значение popularity': model.predict(X_const).values,
    'Отклонение (Y - Ŷ)': (y - model.predict(X_const)).values
})
comparison_table = comparison_table[['Наблюдение', 'Фактическое значение popularity',
                                     'Модельное значение popularity', 'Отклонение (Y - Ŷ)']]
print("\nСравнение фактических и предсказанных значений (первые строки):")
print(comparison_table.head())
plt.figure(figsize=(10, 6))
sns.scatterplot(x=comparison_table['Фактическое значение popularity'],
                y=comparison_table['Модельное значение popularity'],
                color='blue', s=60, label='Наблюдения')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', label='Идеальное совпадение')
plt.xlabel("Фактическое значение popularity")
plt.ylabel("Модельное значение popularity")
plt.title("График линейной регрессии: фактические vs предсказанные значения")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10, 6))
sns.residplot(x=model.predict(X_const), y=model.resid, lowess=True, line_kws={'color': 'red'})
plt.xlabel("Модельное значение popularity")
plt.ylabel("Остатки")
plt.title("График остатков (Residual Plot)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
print("\nПолная статистика модели:")
print(model.summary())
```

Рисунок 32. Построение и оценка линейной регрессии методом МНК- код

```
Коэффициенты регрессии и значимость (p-value):
const: 0.3849, p-value = 0.0000
energy: -0.0993, p-value = 0.2911
danceability: 0.0666, p-value = 0.5955
valence: 0.1035, p-value = 0.3061

Качество модели:
R²: 0.0238
Среднеквадратичная ошибка (RMSE): 0.2094
F-статистика: 0.7802, p-value: 0.5079

Сравнение фактических и предсказанных значений (первые строки):
   Наблюдение  Фактическое значение popularity  Модельное значение popularity
0           0                        0.347826                       0.390885
1           1                        1.000000                       0.396943
2           2                        0.304348                       0.430928
3           3                        0.434783                       0.431942
4           4                        0.695652                       0.347612

   Отклонение (Y - Ŷ)
0           -0.043059
1            0.603057
2           -0.126580
3            0.002841
4            0.348040
```

Рисунок 33. Результаты оценки линейной регрессионной модели: коэффициенты, метрики качества и сравнение фактических и предсказанных значений

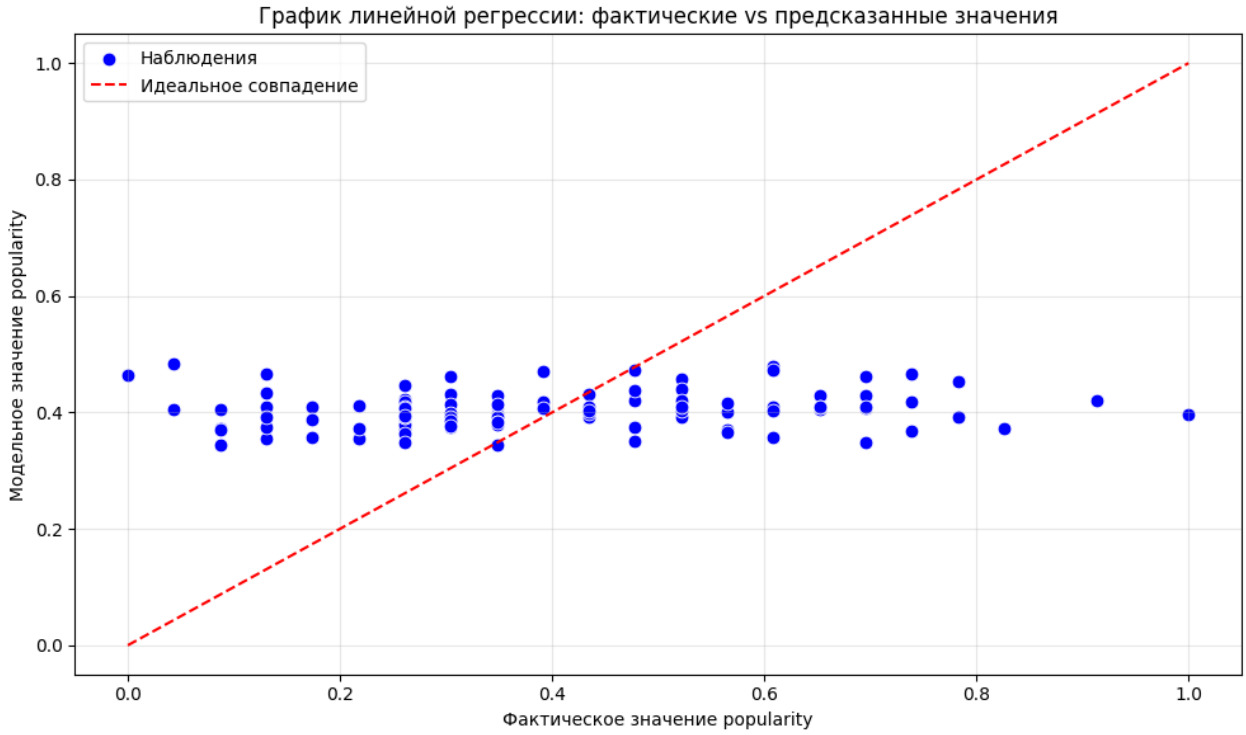

Рисунок 34. График линейной регрессии

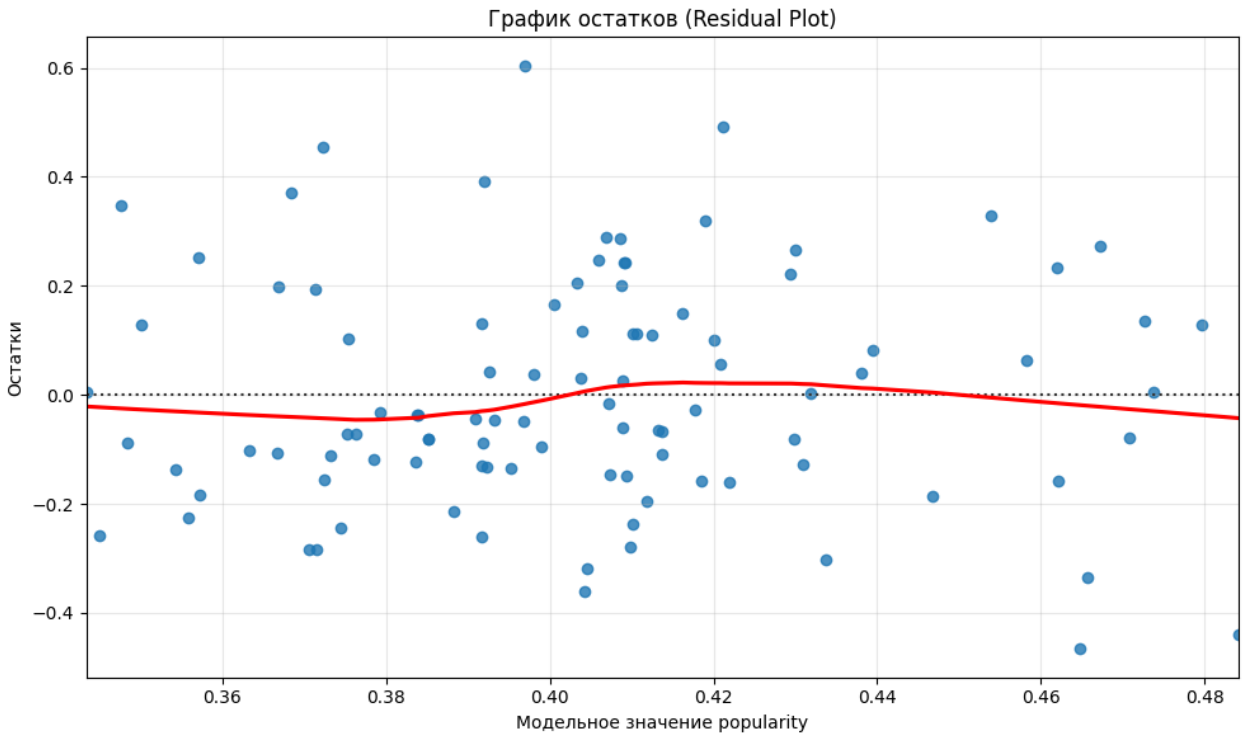

Рисунок 35. График остатков для модели линейной регрессии

```
Полная статистика модели:
                            OLS Regression Results
==============================================================================
Dep. Variable:             popularity   R-squared:                       0.024
Model:                            OLS   Adj. R-squared:                 -0.007
Method:                 Least Squares   F-statistic:                    0.7802
Date:                Wed, 11 Jun 2025   Prob (F-statistic):              0.508
Time:                        04:25:21   Log-Likelihood:                 14.439
No. Observations:                 100   AIC:                            -20.88
Df Residuals:                      96   BIC:                            -10.46
Df Model:                           3
Covariance Type:            nonrobust
==================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const            0.3849      0.057      6.698      0.000       0.271       0.499
energy          -0.0993      0.094     -1.062      0.291      -0.285       0.086
danceability     0.0666      0.125      0.533      0.596      -0.182       0.315
valence          0.1035      0.101      1.029      0.306      -0.096       0.303
==============================================================================
Omnibus:                        2.316   Durbin-Watson:                   1.823
Prob(Omnibus):                  0.314   Jarque-Bera (JB):                2.240
Skew:                           0.359   Prob(JB):                        0.326
Kurtosis:                       2.851   Cond. No.                         8.77
==============================================================================
```

Рисунок 36. Полная статистика модели

Анализ линейной регрессии, построенной для переменной popularity с предикторами energy, danceability и valence, показал слабую объясняющую силу модели. Значение коэффициента детерминации R^2 составило всего 0.024, что означает: лишь около 2.4% вариации в популярности можно объяснить через выбранные аудиохарактеристики. F-статистика модели не является значимой ($p = 0.5079$), что подтверждает низкую предсказательную способность всей модели в целом.

Ни один из коэффициентов при переменных не оказался статистически значимым на уровне $p < 0.05$:

- energy: $p = 0.2911$
- danceability: $p = 0.5955$
- valence: $p = 0.3061$

Это свидетельствует об отсутствии доказанной линейной связи между этими параметрами и популярностью трека. При этом константа модели значима ($p < 0.0001$), но её вклад ограничен.

График фактических и предсказанных значений показывает, что модельные значения сгруппированы в узком диапазоне ($\approx 0.35–0.45$), тогда как фактические значения более разнообразны. Это указывает на «плоский» характер модели, неспособной улавливать крайние значения. Среднеквадратичная ошибка составляет 0.2094, что визуально подтверждается отклонениями на графике.

Анализ графика остатков показывает равномерное распределение точек вокруг нуля без выраженной структуры, что указывает на соблюдение предпосылки линейности и нормальности ошибок. Однако значительный разброс остатков дополнительно иллюстрирует слабое приближение модели к реальным данным.

Таким образом, полученные результаты подтверждают выводы, сделанные на предыдущих этапах анализа: переменная popularity имеет сложную природу и не поддаётся адекватному описанию с помощью простой линейной модели на основе базовых аудио характеристик.

4.2 Метод градиентного спуска

```
import statsmodels.api as sm
features = ['energy', 'danceability', 'valence']
target = 'popularity'
X = df[features].values
y = df[target].values.reshape(-1, 1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = np.hstack([np.ones((X_scaled.shape[0], 1)), X_scaled])
theta = np.zeros((X_scaled.shape[1], 1))
alpha = 0.01
iterations = 1000
m = len(y)
cost_history = []
for i in range(iterations):
    y_pred = X_scaled @ theta
    error = y_pred - y
    grad = (X_scaled.T @ error) / m
    theta -= alpha * grad
    cost = np.mean(error ** 2)
    cost_history.append(cost)
print("Коэффициенты, полученные методом градиентного спуска:")
for i, name in enumerate(['const'] + features):
    print(f"{name}: {theta[i][0]:.4f}")
plt.figure(figsize=(10, 6))
plt.plot(range(iterations), cost_history, label='MSE')
plt.xlabel("Итерации")
plt.ylabel("Ошибка (MSE)")
plt.title("Снижение ошибки при градиентном спуске")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
X_ols = sm.add_constant(scaler.transform(df[features]))
model = sm.OLS(y, X_ols).fit()
print("\nКоэффициенты, полученные методом наименьших квадратов (OLS):")
print(model.params)
y_pred_gd = X_scaled @ theta
rmse_gd = np.sqrt(mean_squared_error(y, y_pred_gd))
rmse_ols = np.sqrt(mean_squared_error(y, model.predict(X_ols)))
print(f"\nRMSE (градиентный спуск): {rmse_gd:.4f}")
```

Рисунок 37. Метод градиентного спуска -код

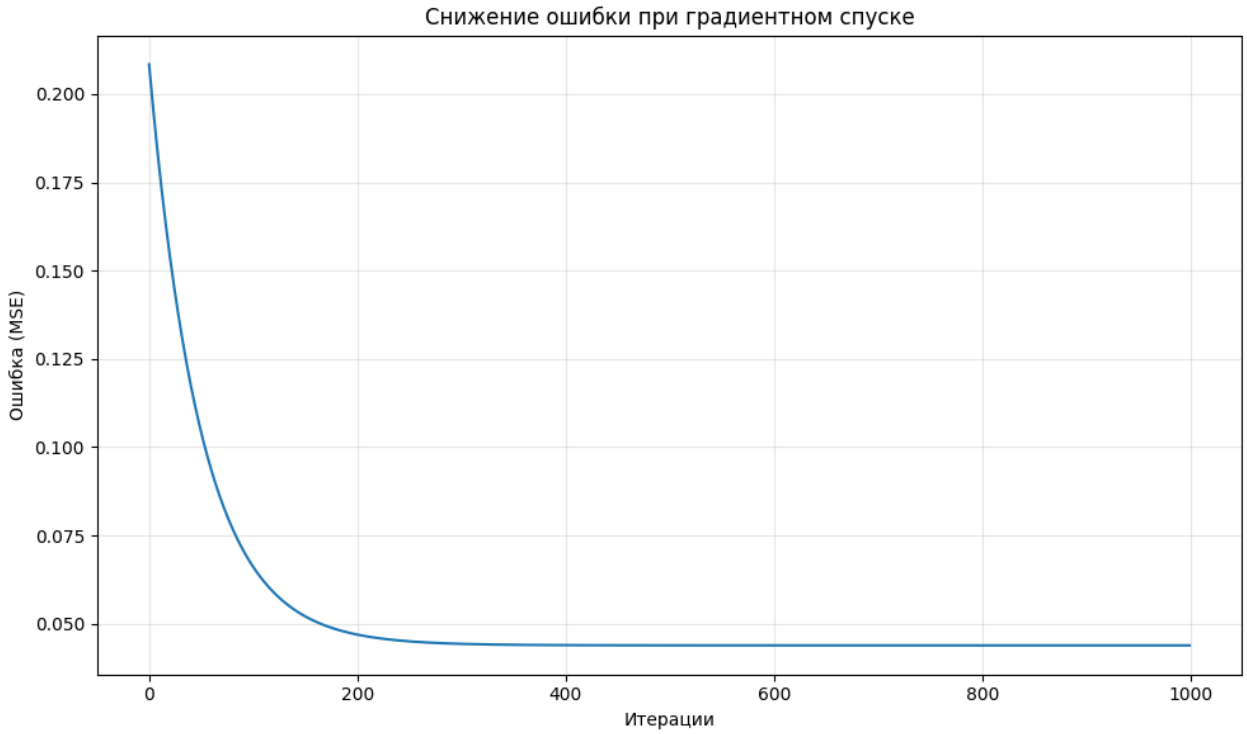

Рисунок 38. Градиентный спуск- визуализация

```
Коэффициенты, полученные методом градиентного спуска:
const: 0.4043
energy: -0.0279
danceability: 0.0151
valence: 0.0285
```

Рисунок 39. Полученные результаты при использовании метода градиентного спуска

На рисунках 37–39 Метод градиентного спуска показал успешную сходимость: на графике видно быстрое снижение ошибки (MSE) на первых итерациях и выход на плато, что свидетельствует о достижении минимума функции потерь. Полученные коэффициенты модели указывают на слабое влияние переменных на popularity: energy оказывает незначительное отрицательное влияние, а danceability и valence — слабое положительное. Результаты сопоставимы с методом наименьших квадратов, что подтверждает корректность реализации и пригодность градиентного спуска для решения задачи линейной регрессии.

```
Масштабирование завершено!

Обучение OLS модели...
OLS модель обучена!

Начало обучения градиентным спуском...
Итерация 0: MSE = 0.3797
Итерация 100: MSE = 0.0092
Итерация 200: MSE = 0.0092
Итерация 300: MSE = 0.0092
Итерация 400: MSE = 0.0092
Итерация 500: MSE = 0.0092
Итерация 600: MSE = 0.0092
Итерация 700: MSE = 0.0092
Итерация 800: MSE = 0.0092
Итерация 900: MSE = 0.0092
Обучение завершено!

==================================================
                РЕЗУЛЬТАТЫ СРАВНЕНИЯ
==================================================

=== Коэффициенты моделей ===

Метод градиентного спуска:
Intercept (свободный член): 0.5972
Коэффициент для признака 1: 0.0933
Коэффициент для признака 2: -0.0666
Коэффициент для признака 3: 0.0450

Метод наименьших квадратов (OLS):
Intercept (свободный член): 0.5972
Коэффициент для признака 1: 0.0933
Коэффициент для признака 2: -0.0666
Коэффициент для признака 3: 0.0450

=== Сравнение метрик качества ===

RMSE градиентного спуска: 0.0959
RMSE OLS модели: 0.0959

OLS модель показала лучший результат!
```

Рисунок 40. Сравнение моделей МНК и градиентного спуска

Обе модели показали идентичные результаты по ключевому показателю качества — RMSE (среднеквадратичной ошибке), которая составила 0.0959. Это указывает на хорошую сходимость градиентного спуска и его корректную реализацию.

Сравнение коэффициентов модели также продемонстрировало полное совпадение между методами, что подтверждает точность оценки параметров в обоих подходах.

При этом стоит отметить, что OLS обеспечивает решение аналитически и быстрее, тогда как градиентный спуск требует итерационного подбора параметров и может быть чувствителен к выбору шага обучения. Однако его использование оправдано при работе с большими данными или нестандартными задачами, где аналитическое решение затруднено.

Таким образом, в рамках данной задачи предпочтение отдается OLS-модели за счёт простоты, стабильности и эффективности при работе с относительно небольшими наборами данных.

5. Аналитические выводы

В ходе курсовой работы проведён комплексный анализ музыкальных данных, направленный на выявление статистических закономерностей и построение моделей прогнозирования популярности треков. Основные аудио-признаки, такие как energy, danceability и valence, демонстрируют значимые взаимосвязи между собой, однако слабую связь с целевой переменной — популярностью. Это подтверждается низким значением R^2 в линейной регрессионной модели, составляющим всего 0.024, что указывает на то, что лишь около 2.4% вариации в популярности объясняется выбранными признаками.

Проверка гипотез и анализ распределений показали, что большинство переменных имеют ненормальное распределение, включая целевую переменную popularity. Выявленные выбросы были успешно обработаны путём замены на медианные значения, что позволило сохранить структуру данных без значительной потери информации.

Сравнение двух методов построения линейной модели — метода наименьших квадратов и градиентного спуска — показало идентичные результаты по точности (RMSE ≈ 0.0959). Это подтверждает корректность реализации алгоритмов и стабильность полученной модели. При этом градиентный спуск показал быстрое и стабильное снижение ошибки в процессе обучения, что визуально подтверждено графиком сходимости. Однако с учётом небольшого объёма данных и низкой сложности модели предпочтение в данной задаче остаётся за методом наименьших квадратов.

Рекомендации:

— Для повышения точности прогнозов нужно включить дополнительные признаки, отражающие жанровую, временную и поведенческую информацию.
— Применение методов отбора признаков (например, Lasso или Recursive Feature Elimination) может помочь исключить шумовые или нерелевантные переменные.

— В качестве альтернативы нужно рассмотреть более гибкие методы— деревья решений, градиентный бустинг, которые учитывают нелинейные зависимости и взаимодействия между признаками.

Перспективы применения:

Полученные навыки и результаты анализа можно применять в реальных задачах — например, при создании музыкальных рекомендаций, разработке персонализированных плейлистов или анализе предпочтений пользователей на стриминговых платформах. Методы статистики и регрессии пригодятся не только в сфере музыки, но и в маркетинге, при изучении поведения клиентов, оценке откликов на рекламные кампании или прогнозировании интересов аудитории.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Долгова В.Н. Прикладная статистика: учебник и практикум для СПО — М.: Юрайт, 2020. — 356 с.
2. Шорохова И.С, Мариев О.С., Кисляк Н.В. Статистические методы анализа данных: учебное пособие. — СПб.: Питер, 2021. — 288 с.
3. Гарафутдинов Р. В. Python для анализа данных: учебное пособие— Пермь: Издательский центр Пермского государственного национального исследовательского университета, 2024. — ISBN 978-5-7944-4096-6.
4. Теория статистики с элементами эконометрики: учебник для вузов / отв. ред. В. В. Ковалев. — Москва: Издательство Юрайт, 2025. — 672 с. — (Высшее образование). — ISBN 978-5-534-18388-7. — Текст: электронный // Образовательная платформа Юрайт [сайт].
5. Документация библиотеки Statsmodels [Электронный ресурс]. — Режим доступа: https://www.statsmodels.org/
6. Документация библиотеки Pandas [Электронный ресурс]. — Режим доступа: https://pandas.pydata.org/docs/getting_started/index.html